UNIVERSIDADE ESTADUAL DE CAMPINAS
Faculdade de Engenharia Elétrica e de Computação

Felipe Augusto Oliveira dos Santos

An Adaptive Defense System for IoT Networks Using Proactive Attack Prediction and Deep Reinforcement Learning

Campinas
2025

Felipe Augusto Oliveira dos Santos

An Adaptive Defense System for IoT Networks Using Proactive Attack Prediction and Deep Reinforcement Learning

Monografia de qualificação apresentada à Faculdade de Engenharia Elétrica e de Computação da Universidade Estadual de Campinas como parte dos requisitos para a obtenção do título de Mestre em Engenharia Elétrica, na área de Engenharia de Computação.

Orientador: Prof. Dr. Denis Fantinato

Campinas
2025

Abstract

The exponential proliferation of the Internet of Things (IoT) has produced a vast network of interconnected devices — the number of worldwide connections is forecast to more than double from 19.8 billion in 2025 to over 40 billion by 2034 — while the global economic impact of cyber threats is projected to reach USD 12.2 trillion annually by 2031. The computational and memory constraints of many IoT devices render traditional, resource-intensive security solutions ineffective, motivating intelligent and adaptive defense mechanisms capable of operating autonomously. This dissertation addresses that need by designing, implementing, and rigorously evaluating an adaptive defense framework for IoT networks that combines proactive attack prediction with Deep Reinforcement Learning (DRL).

The framework is organised as a closed-loop pipeline grounded in the CICIoT2023 dataset, which captures traffic from 105 real IoT devices across 33 attack types. The dataset is projected onto a five-stage Cyber Kill Chain (BENIGN, RECON, ACCESS, MANEUVER, IMPACT), an LSTM-based stage detector and a Long Short-Term Memory red-team episode generator are trained on the resulting stage-labelled splits, and a custom Gymnasium environment exposes a five-action defender (OBSERVE, LOG, THROTTLE, BLOCK, ISOLATE) under a stage-action proportionality reward. Three DRL algorithms — the value-based Deep Q-Network (DQN) and the policy-gradient methods Proximal Policy Optimization (PPO) and Advantage Actor-Critic (A2C) — are trained over five seeds and benchmarked on a held-out test split against four reference baselines: random, always-OBSERVE, always-BLOCK, and a supervised RandomForest stage classifier composed with the kill-chain recommended-action mapping (RF-Acting). A rule-based oracle that observes the true attack stage is reported as an upper bound on the value of perfect stage detection.

The trained agents capture **82 %** of the oracle ceiling without observing the true stage (best deployable agent: DQN, mean episodic reward +1336, 95 % bootstrap CI [1265, 1407], against the oracle's +1624 on the held-out `test_balanced` split) and dominate every non-RL deployable baseline with non-overlapping confidence intervals. A reward-component ablation shows that no single-coefficient 0.5×/2× perturbation closes the residual gap; the gap is instead closed by a structural environment change (`impact_is_terminal`=*False*), which lifts mean reward to +1542 and the mitigated-impact rate from 0.153 to 0.900 (a 5.9× improvement). On a held-out out-of-distribution attack class (`VulnerabilityScan`, on which the supervised stage detector has recall 0.001), the trained policy is *robust to* but not *better at* the OOD class — DQN's mean OOD reward (+1313) lies within seed-noise of its in-distribution mean. Every reported number is reproducible from the per-phase `manifest.json` hash chain and verified end-to-end by the `reproducibility_smoke` harness. The contributions are a kill-chain-aware adaptive defense framework grounded on a contemporary IoT dataset, a comparative DRL benchmark with statistically separated baselines, and an audit-first reproducibility protocol.

Keywords: IoT security, Deep Reinforcement Learning, Cyber Kill Chain, Attack Stage Prediction, LSTM, DQN, PPO, A2C, CICIoT2023, Reproducibility

Contents

1 Introduction and Motivation

1.1 The Case for Adaptive IoT Security

The Internet of Things (IoT) represents a monumental shift in the digital landscape, weaving a network of interconnected devices into the fabric of modern society. This paradigm extends beyond consumer gadgets to critical infrastructure in healthcare, transportation, and industry (Chen *et al.*, 2021). The number of IoT devices worldwide is forecast to more than double from 19.8 billion in 2025 to over 40.6 billion by 2034 (Vailshery, 2025), an exponential growth that has produced a vast and multifaceted attack surface. The financial ramifications are staggering: global damages from cybercrime are projected to reach USD 12.2 trillion annually by 2031 (Morgan, 2025). Malicious actors increasingly exploit insecure devices to steal data, disrupt services, or form large-scale botnets for Distributed Denial of Service (DDoS) attacks (Neto *et al.*, 2023; Tawalbeh *et al.*, 2020).

Securing this landscape presents unique and formidable challenges that render traditional IT security paradigms insufficient. First, most IoT devices are severely **resource-constrained**, possessing minimal computational power, memory, and energy, which precludes the use of intensive security software like traditional firewalls or antivirus agents (Chen *et al.*, 2021). Second, IoT ecosystems are defined by extreme **heterogeneity**, comprising countless devices using disparate protocols and operating systems, which makes uniform security policy enforcement nearly impossible (Mavroeidis, 2023). This complexity is amplified by the large-scale, dynamic nature of these networks and the physical vulnerability of devices often deployed in unsecured locations (Al-Garadi *et al.*, 2020).

Consequently, conventional security tools are fundamentally ill-equipped for this environment. **Signature-based Intrusion Detection Systems (IDS)**, the cornerstone of many legacy systems, rely on static databases of known attack patterns. This approach is purely reactive and fails against novel or **zero-day attacks** — threats for which no signature yet exists (Khraisat *et al.*, 2019; Yang *et al.*, 2024). While **anomaly-based IDS** attempt to overcome this by baselining "normal" behavior, the dynamic and heterogeneous nature of IoT traffic makes establishing a reliable baseline exceptionally difficult, leading to high **false positive rates** that can disrupt critical functions (Ibitoye; Shafiq; Mativenga, 2021; Tharewal *et al.*, 2022).

The core limitation of these paradigms is their lack of adaptability. They are built on static models and cannot autonomously evolve. This creates a critical need for a new secu-

rity paradigm: one that is forward-looking, data-driven, and capable of autonomous decision-making. **Deep Reinforcement Learning (DRL)**, a field of machine learning focused on goal-oriented learning from interaction, has emerged as a powerful framework for such a system (Chen *et al.*, 2021). A DRL agent learns an optimal decision-making strategy, or **policy**, through trial-and-error interaction with its environment. Unlike static systems, a DRL agent can generalise from its experience to respond to novel threats and can continuously adapt its policy as new adversarial tactics emerge, making it well-suited for the non-stationary nature of IoT security (Sutton; Barto, 2018; Uprety; Rawat, 2021). By framing cyber-defense as a sequential decision-making problem, DRL provides a robust mathematical foundation for creating autonomous security systems that can learn to outmaneuver attackers (Gueriani; Kheddar; Mazari, 2023).

1.2 Problem Statement and Contributions

The preceding analysis culminates in a clear research problem: existing IoT security frameworks are not simultaneously **proactive**, **adaptive**, **data-driven**, and **rigorously benchmarked** against deployable reference policies. Reactive signature-based systems cannot anticipate novel attacks, supervised classifiers do not act, and prior DRL-for-IoT work seldom reports head-to-head comparisons against the deployable baselines a security operator actually has at their disposal — random, always-block, and a supervised stage classifier composed with a recommended-action mapping — under a unified evaluation contract.

This thesis directly addresses that gap by designing, implementing, and rigorously evaluating a closed-loop adaptive defense framework for IoT networks that combines proactive attack prediction with Deep Reinforcement Learning, validated end-to-end on the contemporary CICIoT2023 dataset. Concretely, this work makes the following contributions:

1. **A kill-chain-aware proactive-adaptive defense framework.** A two-stage architecture combining (i) a Long Short-Term Memory (LSTM) stage detector and an LSTM red-team episode generator trained on stage-labelled CICIoT2023 sequences with (ii) a custom Gymnasium environment that exposes a five-action defender (OBSERVE, LOG, THROTTLE, BLOCK, ISOLATE) under a stage-action proportionality reward inspired by the IoTWarden recommended-action mapping (Alam; Jahan; Wang, 2024).

2. **Grounding in a realistic, modern dataset.** The framework is developed and vali-

dated on the **CICIoT2023 dataset** (Neto *et al.*, 2023), projected onto a five-stage Cyber Kill Chain (benign, recon, access, maneuver, impact) with a documented per-class mapping (Appendix B). A leakage bug discovered during model training (where four held-out out-of-distribution attack classes had inadvertently leaked 70 % of their rows back into the training pool) was fixed and locked behind disjointness assertions before any benchmark numbers were collected.

3. **A comparative DRL benchmark against deployable baselines and an oracle reference.** DQN, PPO, and A2C are trained over five seeds and evaluated on a held-out balanced test split against four reference policies: random, always-observe, always-block, and a supervised RandomForest stage classifier composed with the recommended-action mapping (RF-Acting). A rule-based policy that observes the true attack stage is reported as an *oracle ceiling* on the value of perfect stage detection, not as a competing baseline. The trained agents capture 82 % of that ceiling on the test split with non-overlapping bootstrap confidence intervals against every non-RL deployable baseline.

4. **Reward-component and out-of-distribution ablations.** A 12-cell sparse one-at-a-time sweep over the five reward coefficients × {0.5×, 1×, 2×} plus a binary environment-semantics flag identifies a structural change to the IMPACT-stage termination contract that closes 71 % of the residual gap to the oracle ceiling and lifts the mitigated-impact rate from 0.153 to 0.900. A separate evaluation against four held-out out-of-distribution attack classes characterises the policy's behaviour on signature-novel inputs and yields the pre-registered finding that the trained policy is robust to, but not better at, the OOD class on which the supervised stage detector is structurally blind.

5. **An audit-first reproducibility protocol.** Every figure ships a `manifest.json` that pins the SHA-256 hashes of every input artefact (data splits, trained models, scaler, episode roll-outs) and the producing Git commit. A smoke-reproducibility harness (`python -m scripts.reproducibility_smoke`) verifies the full hash chain end-to-end on a fresh checkout in approximately five seconds. The protocol is documented as Appendix A.

The remainder of this thesis is structured as follows. Section 2 provides a detailed review of background concepts in IoT security, the Cyber Kill Chain, and reinforcement learning,

and situates this work against related research. Section 3 details the proposed framework architecture, including the kill-chain projection of CICIoT2023, the LSTM red team and stage detector, the Markov Decision Process underlying the environment, the reward function, the training protocol, and the baseline policies. Section 4 presents the empirical results across all phases of the pipeline and discusses the headline findings, including the oracle-ceiling reframe, the reward-component ablation, and the out-of-distribution robustness story. Section 5 summarises the contributions, discusses limitations and threats to validity, and outlines future work.

2 Background and Related Work

This section establishes the theoretical groundwork and contextualises the contributions of this thesis. Section 2.1 introduces the Cyber Kill Chain abstraction adopted as the unit of analysis throughout the dissertation. Section 2.2 reviews the foundational principles of Reinforcement Learning and the specific Deep Reinforcement Learning algorithms central to this work. Section 2.3 introduces the deployable-baseline taxonomy and the role of the rule-based oracle as a measurement instrument. Section 2.4 provides a review of related research in IoT security and DRL-based defense and positions this thesis within that literature.

2.1 The Cyber Kill Chain as a Defender's Decision Frame

The **Cyber Kill Chain** (Mavroeidis, 2023) is a conceptual model that decomposes a sustained intrusion into a sequence of progressively more compromising stages. Originally formulated by industrial threat-intelligence practitioners and refined by the academic community, it organises adversary behaviour as an ordered chain in which earlier stages prepare for the success of later stages and where defender opportunity costs grow monotonically with stage progression: it is much cheaper to interrupt an attack at RECON than at IMPACT.

Following the formulation adopted in this thesis (and consistent with the IoTWarden recommended-action mapping (Alam; Jahan; Wang, 2024)), traffic on an IoT network is projected onto a five-stage kill chain:

1. **BENIGN** — routine, non-malicious traffic. The defender's correct posture is observational.

2. **RECON** — the adversary scans, fingerprints, or otherwise enumerates the network and its assets. Behaviour is malicious but the network is not yet compromised; the defender's correct posture is to log and continue monitoring.

3. **ACCESS** — the adversary establishes an initial foothold (e.g. exploit-driven authentication bypass, brute-force, command injection). The defender should throttle the implicated traffic and prepare to escalate.

4. **MANEUVER** — the adversary performs lateral movement, privilege escalation, or other in-network manipulation to position for a final action. The defender should block.

5. **IMPACT** — the adversary's terminal action: data exfiltration, denial-of-service, ransomware detonation. The defender's last opportunity to prevent loss is to isolate the affected segment.

The kill chain is more than narrative scaffolding: it provides a discrete decision frame in which (i) the attacker's evolving state is summarised by a single ordered variable and (ii) the defender's action menu can be aligned to that variable through a recommended-action mapping (BENIGN → OBSERVE, RECON → LOG, ACCESS → THROTTLE, MANEUVER → BLOCK, IMPACT → ISOLATE). This alignment is the basis of the stage-action proportionality reward used in Section 3.5, and it is what makes the rule-based recommended-action policy a meaningful upper bound (Section 2.3).

2.2 Reinforcement Learning for Autonomous Systems

Reinforcement Learning (RL) is a computational paradigm focused on goal-directed learning from interaction. An intelligent agent learns to make optimal decisions not by being shown labeled examples, but by exploring an environment and discovering which sequences of actions yield the greatest cumulative reward (Sutton; Barto, 2018). This learning process is formalised through the mathematical framework of Markov Decision Processes (MDPs).

2.2.1 Markov Decision Processes

An MDP provides a formal model for sequential decision-making under uncertainty. The core principle is the **Markov Property**, which asserts that the current state provides all necessary information to make an optimal decision, rendering the history of prior states irrelevant (Sutton; Barto, 2018). An MDP is defined as a five-tuple (S, A, P, R, γ):

- S: A finite set of states, representing all possible configurations of the environment. In this thesis, a state $s \in S$ is a multi-modal observation composed of a vector of CICIoT2023 traffic features, a history of recent states and actions, and (optionally) the discrete predicted attack stage produced by the LSTM stage detector.
- A: A finite set of actions. For this work, A is the five-action defender menu derived from the kill chain in Section 2.1: $A = \{\text{OBSERVE}, \text{LOG}, \text{THROTTLE}, \text{BLOCK}, \text{ISOLATE}\}$, ordered by escalating defensive force and operational disruption.
- P: The state transition probability function, $P(s' \mid s, a) = \Pr(S_{t+1} = s' \mid S_t = s, A_t = a)$, defining the dynamics of the environment.
- R: The reward function, $R(s, a, s')$, which provides the immediate numerical feedback for transitioning from s to s' via action a. The full as-built reward used in this thesis is detailed in Section 3.5.
- γ: The discount factor ($0 \leq \gamma \leq 1$), which prioritises immediate rewards over future ones.

The agent's objective is to learn a **policy**, $\pi(a \mid s)$, which maps states to a probability distribution over actions, to maximise the expected discounted return.

2.2.2 Value Functions and Bellman Equations

To determine the quality of states and actions, RL uses value functions. The **state-value function**, $V^\pi(s)$, is the expected return from state s while following policy π:

$$V^\pi(s) = \mathbb{E}_\pi \left[\sum_{k=0}^{\infty} \gamma^k R_{t+k+1} \,\middle|\, S_t = s \right] \tag{1}$$

The **action-value function** (or Q-function), $Q^\pi(s, a)$, is the expected return after taking action a from state s and then following policy π:

$$Q^\pi(s, a) = \mathbb{E}_\pi \left[\sum_{k=0}^{\infty} \gamma^k R_{t+k+1} \,\middle|\, S_t = s, A_t = a \right] \tag{2}$$

These functions adhere to the recursive relationships defined by the Bellman equations. The **Bellman optimality equation** for the Q-function is central to many RL algorithms and is

expressed as:

$$Q^*(s,a) = \mathbb{E}_{s' \sim P(\cdot|s,a)} \left[R(s,a,s') + \gamma \max_{a' \in A} Q^*(s',a') \right] \tag{3}$$

Solving this equation yields the optimal Q-function, $Q^*(s,a)$, from which the optimal policy can be derived by selecting the action with the highest value in each state.

2.2.3 Deep Reinforcement Learning Algorithms

For environments with large or continuous state spaces, like an IoT network, tabular methods are intractable. Deep Reinforcement Learning (DRL) overcomes this by using deep neural networks to approximate the value functions or policies (Chen *et al.*, 2021). This thesis implements and benchmarks three seminal DRL algorithms that represent the primary families of model-free RL.

Deep Q-Networks (DQN). DQN is a value-based, off-policy algorithm that uses a neural network to approximate the action-value function, $Q(s,a;\theta) \approx Q^*(s,a)$ (Mnih; Kavukcuoglu, *et al.*, 2015). To stabilise learning, DQN introduces two critical mechanisms:

- **Experience Replay:** Experiences (s_t, a_t, r_t, s_{t+1}) are stored in a large replay buffer. The network is trained on mini-batches randomly sampled from this buffer, which breaks temporal correlations in the data.
- **Fixed Target Network:** A separate, periodically updated target network is used to generate stable target values for the Bellman equation, preventing oscillations during training. The loss function is the Mean Squared Bellman Error:

$$L(\theta) = \mathbb{E}_{(s,a,r,s') \sim D} \left[\left(\underbrace{r + \gamma \max_{a'} Q(s',a';\theta^-)}_{\text{Target Q-value}} - \underbrace{Q(s,a;\theta)}_{\text{Predicted Q-value}} \right)^2 \right] \tag{4}$$

  where θ^- are the parameters of the fixed target network and $\boldsymbol{D}$ is the replay buffer.

Proximal Policy Optimization (PPO). PPO is a state-of-the-art on-policy actor-critic algorithm that directly optimises a parameterised policy (the actor) while using a value function (the critic) to reduce variance (Schulman *et al.*, 2017). Its key innovation is a clipped surrogate objective function that constrains the size of policy updates, ensuring greater training

stability:

$$L^{\text{CLIP}}(\theta) = \hat{\mathbb{E}}_t \left[\min \left(r_t(\theta)\hat{A}_t,\ \text{clip}(r_t(\theta), 1-\epsilon, 1+\epsilon)\,\hat{A}_t \right) \right] \tag{5}$$

where $r_t(\theta)$ is the probability ratio between the new and old policies, $\hat{A}_t$ is the estimated advantage, and ϵ is a hyperparameter that constrains the policy change. This approach prevents destructive policy updates and is known for its reliability and performance.

Advantage Actor-Critic (A2C). A2C is a synchronous variant of the foundational Asynchronous Advantage Actor-Critic (A3C) algorithm (Mnih; Badia, *et al.*, 2016). Like PPO, it is an actor-critic method, but it uses a simpler, unconstrained policy update mechanism. It typically employs multiple parallel workers to collect experience simultaneously; these experiences are then aggregated to compute a single stable gradient update for both the actor (policy) and the critic (value function). While often faster to converge than PPO on simpler problems, it can be less stable.

All three algorithms are implemented in this thesis using Stable Baselines3 `MultiInputPolicy`, which is designed to handle the dictionary-shaped observation space exposed by the custom IoT environment described in Section 3.5.

2.3 Deployable Baselines and the Role of an Oracle Reference

A defining methodological choice of this work is to evaluate the trained DRL agents not only against *trivial* baselines (random, always-OBSERVE, always-BLOCK) but also against two stronger references that bracket the achievable performance under the kill-chain decision frame:

- **RF-Acting (deployable supervised + rule).** A composite policy that uses a supervised RandomForest classifier (trained in the stage-detection phase) to predict the current attack stage from the observed feature vector, then maps the predicted stage to an action via the recommended-action mapping. RF-Acting is fully deployable: it consumes only what a real defender can observe and produces a defensive action at every step.

- **Recommended-Action rule (oracle reference, *not* a competing baseline).** An identical action-selection rule that, instead of consuming a learned classifier's output, reads the *true* attack stage directly from the simulator's privileged information. This oracle is therefore a measurement instrument that quantifies the value of perfect stage detection

under the chosen reward function. It is reported as an *upper bound* on what any deployable policy could achieve under the recommended-action mapping; it is *not* reported as a method this work claims to outperform, because the oracle does not run on observable inputs and cannot be deployed.

The trained DRL agents are deployable in the same sense as RF-Acting — they observe a vector of CICIoT2023 traffic features and (optionally) the predicted stage, and produce an action — and are scored against both deployable baselines and against the oracle ceiling. Reporting the oracle as a measurement instrument rather than a competing baseline is a methodological commitment that frames the headline thesis claim as "trained RL captures X % of the value of perfect stage detection" rather than the weaker and potentially misleading "trained RL beats the recommended-action policy". This choice was retroactively endorsed by the audit-first review protocol in this work and is documented in detail in Section 4.

2.4 Related Work in AI-Driven IoT Security

The application of artificial intelligence to IoT security is an active and rapidly evolving field of research. Early work focused on applying classical machine learning models to intrusion detection, which has since paved the way for more sophisticated, adaptive systems based on Deep Reinforcement Learning.

2.4.1 Machine Learning Approaches for Intrusion Detection

Early approaches to intelligent IoT security leveraged supervised and unsupervised machine learning for building Intrusion Detection Systems (IDS). Supervised models, such as Support Vector Machines and Random Forests, were trained on labeled datasets to classify network flows as either benign or malicious (Al-Garadi *et al.*, 2020). While effective at identifying known attack patterns, these models struggle with novel or **zero-day attacks** not seen during training, a limitation they share with traditional signature-based IDS (Khraisat *et al.*, 2019). Furthermore, they typically operate as passive classifiers, identifying threats without the capacity for automated, sequential defensive actions — a limitation this thesis addresses directly by composing the supervised classifier with a recommended-action rule (RF-Acting) and using it as a deployable baseline against which the DRL agents are scored.

2.4.2 The Shift to Active Defense with DRL

The limitations of static ML models motivated a shift towards DRL to create autonomous agents capable of active, closed-loop cyber-defense. Several notable works have explored this domain and provide the foundation upon which this thesis builds.

Some research has focused on using RL for deception and intelligence gathering. The work of Guan et al. in **HoneyIoT** (Guan *et al.*, 2023) presents an adaptive high-interaction honeypot that uses RL to learn an attacker's behavior and dynamically alter its services to keep the attacker engaged and reveal their methods. While this is a powerful application of RL, its primary purpose is reconnaissance, whereas the system presented in this thesis is designed as a direct defense mechanism for a live network.

Other works have targeted specific layers of the IoT ecosystem. **IoTWarden**, proposed by Alam et al. (Alam; Jahan; Wang, 2024), uses a DQN agent to mitigate attacks in trigger-action IoT platforms (e.g. IFTTT) by identifying and disabling malicious rules or "applets" in real time. This thesis adopts the IoTWarden *recommended-action mapping* (BENIGN → OBSERVE, RECON → LOG, …, IMPACT → ISOLATE) as a design choice for the environment's reward function, and treats IoTWarden as an inspiration rather than a head-to-head baseline: IoTWarden was evaluated on a different (synthetic, trigger-action) environment with a different action space, so direct numerical comparison is not methodologically sound. The headline numerical claim of this thesis is therefore framed against the in-domain oracle ceiling described in Section 2.3, never against an external IoTWarden number.

The work by Nguyen et al. (Nguyen *et al.*, 2021) proposes a Federated DRL system for traffic monitoring in Software-Defined Networks for IoT, focusing on efficient data collection and analysis rather than active defense. Tharewal et al. (Tharewal *et al.*, 2022) apply DRL to industrial IoT intrusion detection but treat the agent as a classifier rather than a sequential decision-maker. The research in this thesis addresses a more fundamental challenge by operating at the **network layer**, analysing features derived from raw traffic to defend against a broad range of network-level attacks such as DDoS, DoS, and Spoofing, which are prevalent in the CICIoT2023 dataset, and by structuring the policy as a kill-chain-aware sequential decision-maker.

2.4.3 Positioning the Thesis: Bridging the Research Gap

While previous works have successfully applied DRL to specific aspects of IoT security, a gap remains for a comprehensive, network-level defense system that is simultaneously:

- **Proactive and Adaptive:** combining a supervised stage detector and an LSTM-based red-team episode generator with a DRL-based agent in a closed loop from threat anticipation to autonomous action.
- **Grounded in Realistic, Modern Data:** validated on the large-scale and contemporary **CICIoT2023 dataset**, with documented kill-chain projections and a fully audited splits protocol that closes a subtle out-of-distribution leakage bug found during model training.
- **Network-Layer Focused:** designed to defend against a broad range of fundamental network attacks by analysing traffic-flow features.
- **Rigorously Benchmarked:** providing a thorough empirical comparison of modern value-based (DQN) and actor-critic (PPO, A2C) algorithms against three trivial baselines, a deployable supervised + rule baseline (RF-Acting), and an oracle reference — with non-overlapping bootstrap confidence intervals as the statistical-separation criterion.
- **Reproducible by Construction:** every figure ships with a SHA-256 hash chain over its inputs and a smoke-reproducibility harness verifies the chain end-to-end on a fresh checkout.

This thesis directly addresses these gaps by designing, implementing, and evaluating a complete framework that integrates these critical components, thereby advancing the development of resilient, intelligent systems capable of defending the vast and ever-expanding IoT landscape.

3 Methodology

This section details the methodology used to design, implement, and evaluate the proposed adaptive IoT defense system. The framework is a multi-stage closed-loop pipeline that synergises predictive modelling with autonomous decision-making to produce a proactive security posture. Section 3.1 presents the high-level architecture. Section 3.2 describes the CICIoT2023

dataset and the kill-chain projection that turns per-flow labels into episode trajectories. Section 3.3 introduces the LSTM-based red-team episode generator, and Section 3.4 the supervised stage-detection module. Section 3.5 formally specifies the Markov Decision Process underlying the custom Gymnasium environment, including the as-built reward function. Section 3.6 details the blue-team training protocol, and Section 3.7 defines the deployable baselines and the oracle reference. Section 3.8 describes the reward-component and out-of-distribution ablations. Section 3.9 introduces the audit-first reproducibility framework that pins every figure to the SHA-256 hash chain of its inputs.

3.1 Framework Architecture Overview

The proposed framework is designed to transition IoT security from a reactive to a proactive paradigm. This is achieved through the architecture illustrated in Figure 1, which can be understood as a closed-loop pipeline of two offline preparation stages and one online learning loop.

The pipeline has three stages:

1. **Stage I — Predictive Foundation (offline).** The CICIoT2023 dataset is projected onto a five-stage kill chain (Section 3.2). Two LSTM-based components are trained on the resulting stage-labelled splits: a *red-team episode generator* that samples next-stage transitions for episode roll-outs (Section 3.3), and a *stage detector* that classifies a per-flow feature vector into one of the five kill-chain stages (Section 3.4). A supervised Random-Forest classifier is also trained as a deployable baseline (Section 3.7).

2. **Stage II — Adaptive Defense System (online training).** A custom Gymnasium environment instantiates the kill-chain MDP (Section 3.5); the red-team LSTM drives stage transitions and a split-aware feature realiser samples observed feature vectors from the appropriate dataset partition. Three DRL algorithms (DQN, PPO, A2C) are trained against this environment (Section 3.6).

3. **Stage III — Benchmarking and Ablation.** The trained checkpoints are evaluated on the held-out `test_balanced` split against four deployable baselines and an oracle reference (Section 3.7); reward-component and out-of-distribution ablations characterise the policy's sensitivity to environment design (Section 3.8).

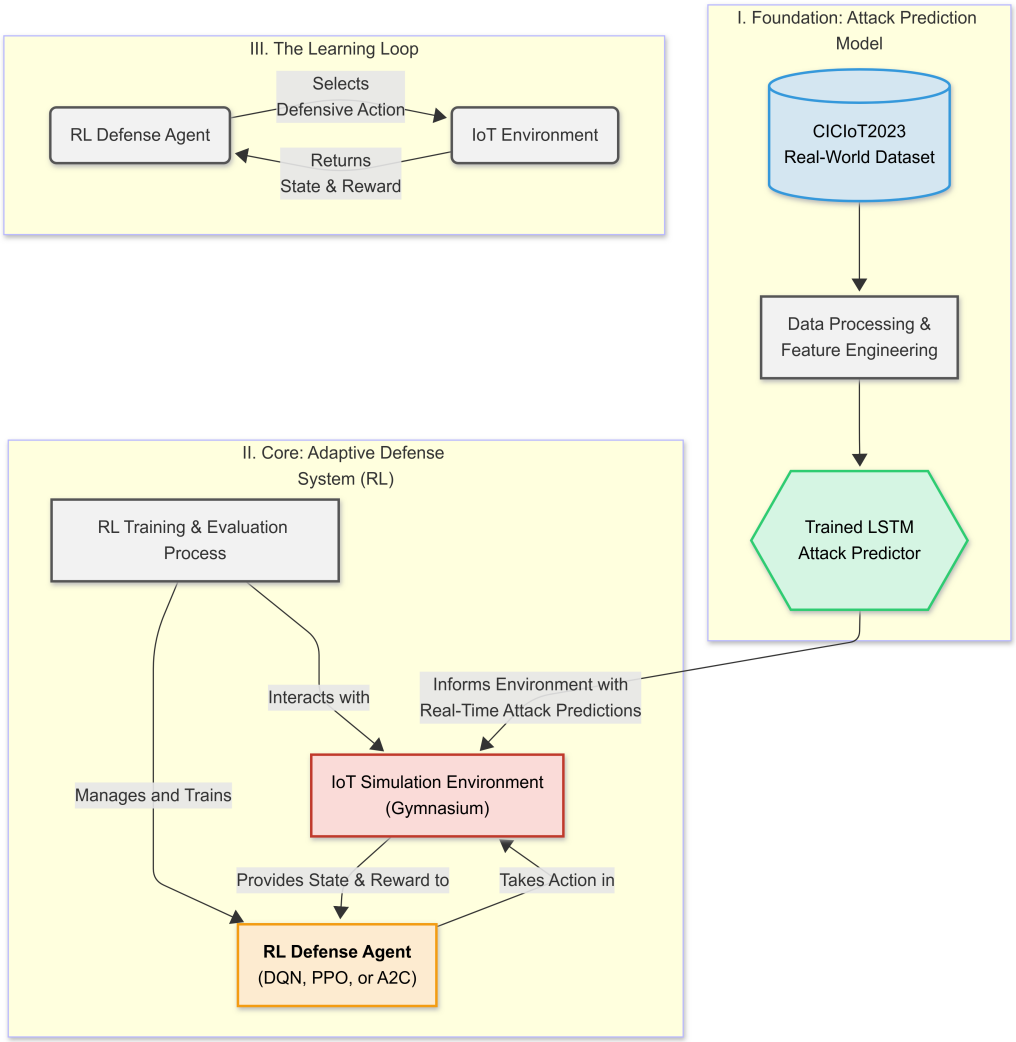


Figure 1: High-level architecture of the proactive-adaptive defense system. The dashed boxes are offline (Phase 1 dataset preparation, Phase 2 red-team LSTM training, Phase 4 stage-detector training); the solid box is the online RL loop (Phase 3 environment, Phase 5 agent training, Phase 6/7 evaluation).

The pipeline is decoupled by phase, with each phase emitting an immutable record under `docs/results/<phase>/` that pins the SHA-256 hash of every input artefact (Section 3.9). This discipline allows the empirical record to be re-verified end-to-end on a fresh checkout in seconds, without retraining any model.

3.2 Data Source and Kill-Chain Projection

The empirical validity of any AI-driven security system depends critically on the quality and realism of its training data. This research uses the **CICIoT2023 dataset** (Neto *et al.*, 2023), a contemporary and extensive resource for IoT security research that addresses the limitations of older synthetic benchmarks.

3.2.1 The CICIoT2023 Dataset

CICIoT2023 was developed by the Canadian Institute for Cybersecurity to provide a realistic benchmark reflecting modern IoT network traffic. Its key characteristics are:

- **Realistic testbed:** traffic was captured from a physical topology of 105 real IoT devices, including smart cameras, speakers, sensors, and hubs (Neto *et al.*, 2023).
- **Comprehensive attack coverage:** the dataset contains benign traffic alongside labelled data for 33 distinct attack types, grouped into seven major categories: DDoS, DoS, Reconnaissance, Web-based, Brute Force, Spoofing, and Mirai variants.
- **Modern attack vectors:** attacks were executed *by* other compromised IoT devices within the testbed, simulating realistic botnet and device-to-device attack scenarios.

3.2.2 Kill-Chain Projection

A per-class mapping projects each of the 34 CICIoT2023 labels (33 attack classes plus benign) onto exactly one of the five kill-chain stages (BENIGN, RECON, ACCESS, MANEUVER, IMPACT) introduced in Section 2.1. The full table is provided in Appendix B; the design principle is that classes describing scanning or fingerprinting (e.g. `Recon-OSScan`, `Recon-PortScan`) map to RECON, classes describing exploit-driven foothold establishment (`BrowserHijacking`, `CommandInje` brute-force variants) map to ACCESS, lateral-movement and spoofing classes map to MANEUVER, and direct denial-of-service or data-loss classes (most DDoS, DoS, and Mirai variants) map to IMPACT. The kill-chain stage is the only label the downstream RL pipeline consumes; the original 34-class label is retained only for the supervised stage-detector evaluation.

3.2.3 Splits Protocol and Out-of-Distribution Reservation

Four attack classes are reserved as held-out *out-of-distribution* (OOD) classes that the supervised models and RL agents never see during training: `DDoS-HTTP_Flood` (IMPACT), `Mirai-udpplain` (IMPACT), XSS (ACCESS), and `VulnerabilityScan` (RECON). The remaining 30 classes are stratified-split into `train` (70 %), `val_balanced` (15 %), and `test_balanced` (15 %) partitions with disjoint indices. The disjointness of the four sets (`train`, `val_balanced`, `test_balanced`, ood) is locked behind explicit assertions in `tests/test_build_split_indices.p`

A subtle leakage bug in the original splits-construction script was discovered during stage-detector training: the OOD classes were correctly identified as held-out but were not removed

from the `train`/`val`/`test` index pools, so 70 % of every "held-out" class was simultaneously a training row. The fix (commit `3cd2fb9`) computes the OOD indices first, masks them out of the index pool, and only then performs the stratified split; the disjointness assertions above were added in the same commit. The leakage bug and its fix are presented honestly in the results chapter (Section 4, §4.2) as a teachable example of the audit-first protocol catching a real bug before it could contaminate downstream phases.

3.3 Red-Team Episode Generator

The training-time environment requires a generative model of attacker behaviour that produces realistic stage-to-stage transitions. The red-team component is a single-layer Long Short-Term Memory (LSTM) network with hidden size 128 trained for 15 epochs with early stopping on the validation cross-entropy loss (Hochreiter; Schmidhuber, 1997). The network is conditioned on a prefix of past stages and predicts the next stage as a softmax distribution over the five kill-chain stages.

The training data are stage-token sequences derived from per-attack-class sequence priors built from the `train` split: each attack class contributes its empirical attacker behaviour to a class-conditional transition matrix, and episodes are generated by composing those priors. The resulting LSTM is upper-triangular by construction — once the chain advances to a more compromising stage, it does not return absent defender intervention — which is consistent with the IoTWarden threat model. Section 4, §4.1 presents the validation curves and the empirical 5×5 transition matrix the trained LSTM generates.

3.4 Supervised Stage Detector

The supervised stage detector classifies a 29-feature CICIoT2023 vector into one of the five kill-chain stages. It is a small multi-layer perceptron with approximately 4,357 parameters (20 KB on disk) and a median per-sample CPU inference latency of 0.039 ms. A 100-tree RandomForest is trained on the same data as a complementary baseline; the RandomForest achieves a higher in-distribution macro-F1 (0.90 vs. 0.79 for the production detector) but is approximately three orders of magnitude larger on disk and is therefore retained primarily as the supervised classifier underlying the deployable RF-Acting baseline (Section 3.7). A 1D-CNN baseline trained on the same data is retained for completeness.

The detector is trained on the post-leakage-fix `train` split (281,420 rows after OOD reservation; pre-fix the figure was 309,566 rows) and evaluated on the held-out `test_balanced` split. Per-stage recall is reported as the principal in-distribution metric and per-class recall as the principal out-of-distribution metric. The detector's behaviour on the four held-out OOD classes is the upstream input to the RL-level robustness story (Section 3.8, F15 ablation): the same OOD-asymmetry pattern (one class with recall 0.001 and three classes with recall ≥ 0.79) recurs in the RL evaluation in a way that is best understood through the supervised result first.

3.5 Reinforcement Learning Environment

The RL environment is a custom Gymnasium task that instantiates the kill-chain MDP. Its core components are the state space, the five-action defender, and the as-built piecewise reward function. Together they implement the formal MDP (S, A, P, R, γ) introduced in Section 2.2.1.

3.5.1 State and Action Spaces

The environment uses a `Dict` observation space that gives the agent a multi-modal view of the network state. At each step the observation contains:

- `current_state`: the 29-dimensional CICIoT2023 traffic feature vector at the current step, sampled by a split-aware realisation engine that draws from the named partition (`train` during training; `test_balanced` during evaluation) and never from any OOD-reserved row.
- `state_history`: a w-step buffer of recent feature vectors, allowing the agent to perceive temporal trends.
- `action_history`: a buffer of the agent's most recent actions, providing self-awareness of recent behaviour.
- `stage_pred` (optional, evaluation-time only): the discrete predicted stage from a frozen supervised classifier, used by the deployable RF-Acting baseline; the trained DRL agents do not consume this field during training.

The action space is the ordered five-action defender menu introduced in Section 2.2: $A = \{\textsc{observe} = 0, \textsc{log} = 1, \textsc{throttle} = 2, \textsc{block} = 3, \textsc{isolate} = 4\}$. The ordering matters

because the reward function below penalises actions outside a ±1 band of the recommended action for the current stage.

3.5.2 Episode Lifecycle

The episode begins at $t = 0$ in the BENIGN stage. At each step, the agent observes the state and chooses an action; the environment then advances the attacker stage according to one of three rules:

1. **Defender-driven de-escalation** (probability $p_{\text{de-esc}} = 0.6$). If the agent picks BLOCK or ISOLATE on an active ACCESS or MANEUVER stage, with probability $p_{\text{de-esc}}$ the env resets the stage to BENIGN and grants the agent a `defense_success_bonus` of +250.

2. **Natural LSTM transition** (otherwise). The red-team LSTM samples the next stage from its conditional distribution given the recent transition history.

3. **Lifecycle-floor IMPACT clamp.** If the natural transition produces IMPACT before $t =$ `min_episode_length` $= 20$, the stage is clamped to MANEUVER for that step. This reflects the threat-model assumption that real-world IMPACT requires time-to-execute and is not an instantaneous jump from RECON.

The episode terminates when the chain reaches IMPACT (default Phase-3 contract; this can be flipped by the `impact_is_terminal` flag exercised in the F9 ablation, see Section 3.8) or is truncated at `max_steps` = 500.

3.5.3 Reward Function

The reward at step t is a piecewise composition of **six independent reward signals** plus three asymmetric guardrails for cocked-trigger benign-stage policies. Let a_t denote the action chosen at step t, s_t the decision-time stage (i.e. the stage the agent saw when it chose a_t, not the next stage), and $rec(s)$ the recommended-action mapping (BENIGN → OBSERVE, RECON → LOG,

ACCESS → THROTTLE, MANEUVER → BLOCK, IMPACT → ISOLATE). The per-step reward is:

$$\begin{aligned} R_t \;=\; & -\,C_{\text{action}}(a_t)\cdot\alpha \\ & +\,r_{\text{prop}}\cdot[|a_t - \mathit{rec}(s_t)| \le 1] - p_{\text{disp}}\cdot[|a_t - \mathit{rec}(s_t)| \ge 2] \\ & +\,r_{\text{benign}}\cdot[s_t = \textsc{benign} \;\wedge\; a_t \le \textsc{log}] \\ & -\,p_{\text{over}}\cdot[s_t = \textsc{benign} \;\wedge\; a_t \ge \textsc{throttle}] \\ & -\,p_{\text{block-benign}}\cdot[s_t = \textsc{benign} \;\wedge\; a_t \ge \textsc{block}] \\ & -\,p_{\text{block-recon}}\cdot[s_t = \textsc{recon} \;\wedge\; a_t \ge \textsc{block}] \end{aligned} \tag{6}$$

At terminal-IMPACT, an additional accounting block fires:

$$R^{\text{term}} \;=\; -p_{\text{impact}} \;+\; \begin{cases} +b_{\text{success}} & \text{if } a_t \ge \textsc{block} \\ -p_{\text{missed}} & \text{if } a_t \le \textsc{log} \\ 0 & \text{otherwise} \end{cases} \tag{7}$$

Defender-driven de-escalations award $+b_{\text{success}}$ mid-episode when they fire (independent of the terminal-IMPACT step).

The Phase-3 default constants used throughout the dissertation (`AdversarialEnvConfig`) are listed in Table 1. They were calibrated so that the recommended-action policy is net-positive in expectation across an episode (gate G3.4) while the always-OBSERVE policy is net-negative (gate G3.5), with all 13 calibration tests in `tests/test_phase3_env_gates.py` green at the locking commit `36fec22`.

The reward is evaluated against the *decision-time* stage s_t, not against the next stage; this preserves causal credit assignment. Mean Time To Compromise (MTTC), defined as the number of steps from episode start to the first IMPACT transition (with the lifecycle-floor clamp described above), is exposed as a per-episode telemetry signal but is *not* part of the reward function.[1]

[1]Because the lifecycle-floor IMPACT clamp downgrades pre-step-20 IMPACT transitions to MANEUVER, the empirical MTTC distribution is biased toward exactly `min_episode_length = 20` when compromise occurs at all. Mean MTTC values reported in this dissertation should be read as "lifecycle-floor-bounded" rather than "unconstrained natural" MTTC; see `docs/results/03_env/RESULTS.md` §7 R2 for the full caveat.

Table 1: Phase-3 default reward constants (`AdversarialEnvConfig`). Symbols match Equations (6)–(7).

Symbol	Value	Description
α	1.0	action-cost scale
r_{prop}	5.0	per-step proportionality bonus ($\lvert a - rec(s)\rvert \leq 1$)
p_{disp}	5.0	per-step disproportionate-action penalty ($\lvert a - rec(s)\rvert \geq 2$)
r_{benign}	10.0	benign-passive bonus (BENIGN + OBSERVE/LOG)
p_{over}	50.0	overreact-on-benign penalty (BENIGN + $a \geq$ THROTTLE)
$p_{\text{block-benign}}$	100.0	additional block-on-benign penalty
$p_{\text{block-recon}}$	50.0	block-on-recon penalty
p_{impact}	200.0	terminal-IMPACT penalty (always applied)
b_{success}	250.0	defense-success bonus (terminal BLOCK/ISOLATE or de-esc)
p_{missed}	150.0	terminal-IMPACT miss penalty (action $\leq$ LOG)
$p_{\text{de-esc}}$	0.6	defender-driven de-escalation success probability

3.6 Blue-Team Training Protocol

The trained-RL component of the framework consists of three DRL algorithms (DQN, PPO, A2C; Section 2.2.3) trained against the Phase-3 environment. The implementation uses Stable Baselines3 with the `MultiInputPolicy` architecture, which natively handles the `Dict` observation space.

Each algorithm is trained for 250,000 environment steps over five random seeds (one subprocess per (algorithm, seed) pair, total 15 runs). The choice of 250,000 rather than 500,000 was made on the basis of a probe-driven scaling study during Phase 5: PPO mean reward climbed from 497 to 1071 across five 10,000-step buckets and the marginal return became negligible past the 250,000-step horizon, while seed-confidence-interval bands at 250,000 were already publication-tight (±50 reward). Total wallclock for the 15-run sweep is approximately 109 minutes on a single CPU core. Hyperparameters per algorithm (learning rate, batch size, update horizon, network architecture) are listed in Appendix C.

Training is driven against the `train` split with `exclude_ood=True` hardcoded (preventing OOD-reserved rows from leaking into training); evaluation is on the `val_balanced` split during training and on `test_balanced` for the final benchmark in Section 3.7. The eval-distribution contract is verified end-to-end in code; see Section 3.9.

3.7 Baseline Policies and the Oracle Reference

The trained DRL agents are evaluated against four reference policies that span the achievable performance under the kill-chain decision frame, plus an oracle reference that bounds the value of perfect stage detection. All five run on the same evaluation harness as the trained RL policies (deterministic 150-episode roll-outs on `test_balanced`).

- **Random.** Uniform sampling from the five-action menu at every step. The lower bound on any non-trivial policy.
- **Always-OBSERVE.** A purely passive policy that picks OBSERVE at every step regardless of stage. The cost-only baseline: zero defensive force, maximum operational throughput, every IMPACT lands.
- **Always-BLOCK.** A policy that picks BLOCK at every step. The reverse of always-OBSERVE: maximum defensive force at the cost of guaranteed operational disruption.
- **RF-Acting (deployable).** The composite policy that uses a frozen supervised Random-Forest classifier to predict the current stage from the observed feature vector and then maps the predicted stage to an action via the recommended-action mapping. RF-Acting is fully deployable; its strength is bounded by the supervised classifier's accuracy.
- **Recommended-Action rule (oracle reference, *not* a competing baseline).** The same action-selection rule but consuming the *true* attack stage from the simulator's privileged `info["attack_stage"]` field. This is an oracle: a real defender cannot read the simulator's state. Reported throughout this dissertation as an upper bound on the value of perfect stage detection under the chosen reward function.

3.8 Reward-Component and Out-of-Distribution Ablations

Two ablation strands characterise the trained policy's sensitivity to environment design.

3.8.1 Reward-Component Sweep (F9)

A sparse one-at-a-time sweep perturbs each of the five principal reward coefficients (`defense_success_`
`penalty_missed_impact`, `reward_proportional`, `reward_benign_passive`, `penalty_dispr`
by 0.5× and 2.0× around its Phase-3 default, holding all other coefficients fixed. A separate cell

perturbs the binary `impact_is_terminal` flag, which controls whether the IMPACT-stage transition immediately ends the episode (default *True*) or grants the agent one final mitigation turn (*False*). Each cell trains PPO over five seeds for 250,000 steps under the perturbed environment and reports mean test reward + bootstrap CI on `test_balanced`.

The verdict logic for the sweep is two-strand to remain apples-to-apples (Phase-7 §5.2 D7.1.1): cells that scale a reward coefficient also scale the absolute reward number, so they are not directly comparable to the Phase-6 unscaled baseline; only cells that preserve the Phase-3 reward function (the centre baseline and the two `impact_is_terminal` cells) are eligible for the apples-to-apples raw-reward gate. Reward-coefficient cells are scored on the `mitigated_impact_rate` security KPI, which is unaffected by reward-coefficient scaling.

3.8.2 Aggressiveness Sweep (F10)

A six-point sweep over the defender de-escalation probability $p_{\text{de-esc}} \in \{0.0, 0.2, 0.4, 0.6, 0.8, 1.0\}$ characterises the trained policy's response to attacker aggressiveness, with the oracle recommended-action rule run as a reference at every p value.

3.8.3 Out-of-Distribution Robustness (F15)

The four held-out OOD attack classes (`DDoS-HTTP_Flood`, `Mirai-udpplain`, `XSS`, `VulnerabilitySc` are evaluated against every Phase-6 policy (DQN, PPO, A2C × 5 seeds, plus the four baselines and the oracle rule). The evaluation uses a *hybrid realiser*: an in-distribution feature pool drawn from the post-leakage-fix `train` split is overlaid with the OOD class's rows only at the kill-chain stage that class belongs to (e.g. `VulnerabilityScan` rows replace the RECON stage's natural in-distribution feature realiser; the other four stages are unchanged). This design isolates the OOD signal to the one stage where it matters and avoids forcing `env.reset()` to fail on stages the OOD class does not occupy. The pre-registered finding-activation rule D7.9.1 narrows the thesis claim if the trained RL policies do not beat the deployable RF-Acting baseline on the hardest OOD class; this finding does activate empirically and is presented in Section 4, §4.6.

3.9 Reproducibility Framework

Every figure under `docs/results/<phase>/` ships with a `manifest.json` that records (a) the SHA-256 hash of every input artefact (data splits, trained models, scaler, episode roll-

outs, upstream phase manifests), (b) the producing Git commit, and (c) the producing-script invocation. A smoke-reproducibility harness, `python -m scripts.reproducibility_smoke`, walks the full hash chain in approximately five seconds on a fresh checkout and reports a per-input verdict bucketed into **OK** (hash matches on disk), **FAIL** (hash mismatches; reproducibility broken), **KNOWN-DIVERGENCE** (a documented and audited mismatch, e.g. the pre-leakage-fix splits-manifest SHA recorded by Phase 1 and Phase 2 before the `3cd2fb9` fix; documented in `docs/results/02_red_team/RESULTS.md` §5.3), and **SKIP** (a gitignored input not on disk). The harness's verdict at the head of the dissertation's evaluation chain is **PASS** (458 OK, 0 FAIL, 2 KNOWN-DIVERGENCE, 6 SKIP). The full reproducibility recipe is documented in Appendix A.

4 Results and Discussion

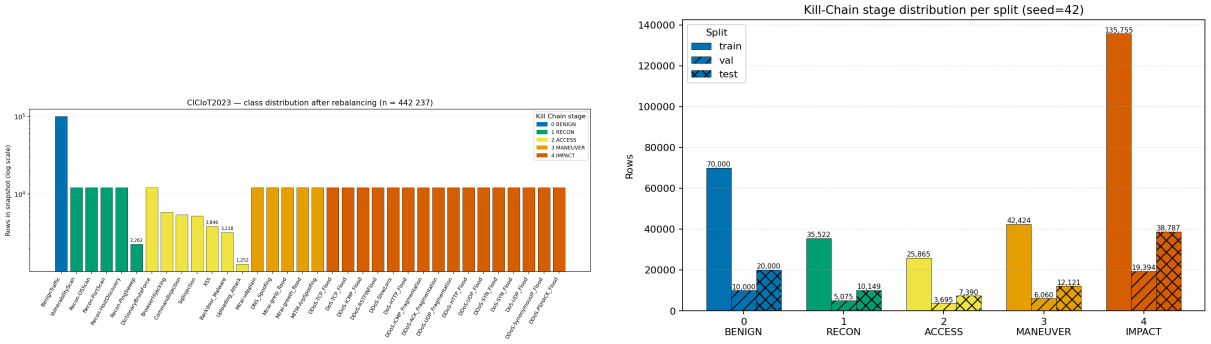
This chapter presents the empirical results across all phases of the framework. Section 4.1 fixes the experimental setup. Section 4.2 reports the kill-chain projection of CICIoT2023 and the splits-protocol audit (including the leakage-bug fix). Section 4.3 validates the LSTM red-team episode generator; Section 4.4 validates the supervised stage detector. Section 4.5 reports the DRL training results, and Section 4.6 the held-out benchmarks against deployable baselines and the oracle reference. Section 4.7 reports the reward-component and aggressiveness ablations, and Section 4.8 the out-of-distribution robustness evaluation. Section 4.9 discusses the headline findings and threats to validity.

4.1 Experimental Setup

The experiments were run on a single CPU core (no GPU is required for any phase) using Python 3.9, PyTorch for the LSTM red-team and the stage detector, scikit-learn for the RandomForest baseline, Stable Baselines3 for the DRL agents, and Gymnasium for the custom environment. The blue-team training sweep takes approximately 109 minutes; the held-out benchmark sweep approximately 10 minutes; the Phase-7 ablations approximately 7.5 hours wallclock. All trained-model and roll-out artefacts are pinned by SHA-256 hash chains (Section 3.9, Appendix A), and the full `pytest` test suite (411 tests) is green at every commit cited in this chapter.

4.2 Dataset Projection and Splits Audit

The CICIoT2023 dataset is projected onto the five kill-chain stages using the per-class mapping documented in Appendix B. Figure 2 shows (a) the per-class distribution after rebalancing and (b) the per-stage distribution across the four splits.



(a) Per-class distribution

(b) Per-stage distribution per split

Figure 2: CICIoT2023 dataset overview after kill-chain projection. Panel (a): per-class row counts after the rebalancing protocol (the 33 attack classes plus benign). Panel (b): per-stage row counts across the four partitions (`train`, `val_balanced`, `test_balanced`, `ood`).

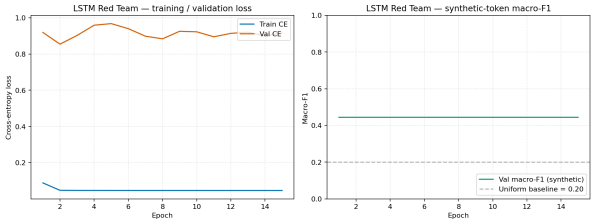
The post-leakage-fix `train` partition contains 281,420 rows (RECON 27,038, ACCESS 23,173, MANEUVER 33,939, IMPACT 127,270, BENIGN balance) after OOD reservation. The 28,146-row delta against the pre-fix counts (309,566 total rows) corresponds exactly to the four held-out OOD attack classes' training fraction (40,209 rows × 0.70 train ratio = 28,146 rows). The disjointness of the four partitions is locked behind explicit assertions in `tests/test_build_split_indic` (`train` ∩ `ood` = `val` ∩ `ood` = `test` ∩ `ood` = ∅); these assertions are part of the 411-test suite and are checked at every commit.

The leakage-bug fix as a methodological contribution. A subtle leakage bug in the original splits-construction script was discovered during the supervised stage-detector training: `train_detector.py` aborted with the message "`LEAKAGE: train ∩ ood:DDoS-HTTP_Flood = 8 546 rows`", triggered by a defensive disjointness check added to the producer script before any model training began. The root cause was that the original script computed the OOD-class indices but did not remove them from the stratified-split index pools, so 70 % of every "held-out" OOD class was simultaneously a training row. The fix (commit `3cd2fb9`) reorders the operations: OOD indices are computed first and masked out of the index pool, and only then are the in-distribution rows stratified-split. The pre-fix `splits/manifest.json` SHA (`82aa1214...`) was preserved in the Phase-1 and Phase-2 `manifest.json` records as the

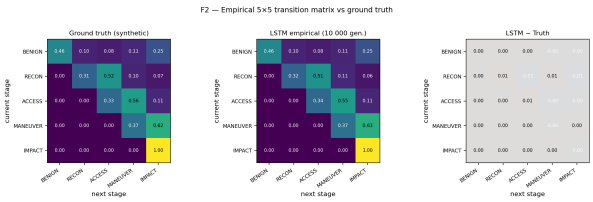
immutable audit-trail anchor; the on-disk post-fix SHA (`1e99d596...`) is what every later phase consumes. Both are surfaced as *KNOWN-DIVERGENCE* cells by the `reproducibility_smoke` harness (Section 3.9) with the explanatory note in `docs/results/02_red_team/RESULTS.md` §5.3. Phase-2 was *not* retrained because the LSTM consumes only stage tokens (not per-flow features), making the leakage orthogonal to its input space; this is verified empirically by the post-fix Phase-2 cosine-similarity gate (G4 saturation at 0.99999, Section 4.3).

4.3 Red-Team Episode Generator

The single-layer LSTM red team is trained for 15 epochs with early stopping on the validation cross-entropy. Figure 3 shows (a) the training and validation curves and (b) the empirical 5×5 stage-transition matrix versus the ground-truth class-conditional priors.



(a) LSTM learning curves



(b) Empirical vs. ground-truth transition matrix

Figure 3: Phase 2 LSTM red-team episode generator. Panel (a): cross-entropy loss and token accuracy on training and validation sets across 15 epochs. Panel (b): empirical 5×5 stage-transition matrix produced by the trained LSTM (left) compared against the ground-truth transition priors built from the `train` split (right).

The Phase-2 exit-gate scoreboard at the locking commit `283ca29e` reads:

- **G1** (in-distribution generalisation, train↔val gap ≤ 0.25): observed **0.035** — PASS.
- **G2** (token accuracy on holdout ≥ 0.55): observed **0.977** — PASS.
- **G3** (KL divergence to ground-truth transition matrix ≤ 0.05): observed **0.021** — PASS.
- **G4** (cosine similarity, LSTM rollouts vs. ground-truth rollouts, ≥ 0.90): observed **0.99999** — PASS.
- **G5** (full pytest suite green): **411/411** at HEAD — PASS.

The G4 cosine saturation at 0.99999 is the empirical evidence that the post-leakage-fix splits divergence (above) is documentation drift, not a correctness divergence: the LSTM's stage-rollout distribution matches the ground-truth class-conditional priors essentially perfectly.

The model-selection criterion was *balanced-validation cross-entropy* (rather than raw validation accuracy) because the per-stage class frequencies are heavily imbalanced; this avoids the trap of selecting a model that is excellent at the dominant BENIGN class and indifferent to the other four. The seed used for all randomness during Phase-2 training is `seed=42`, justified empirically by a 10-seed sensitivity probe documented in `docs/results/02_red_team/RESULTS.md` §5.2.

4.4 Supervised Stage Detector

The supervised stage detector is evaluated on the held-out `test_balanced` split. Figure 4 shows the per-stage recall for the production MLP detector and the two reference baselines (RandomForest and 1D-CNN).

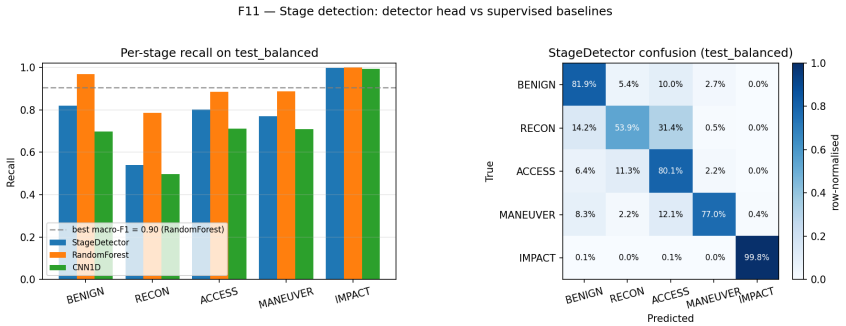


Figure 4: Per-stage recall on the held-out `test_balanced` split for the production MLP stage detector and two reference baselines. RandomForest saturates at the 29-feature CICIoT2023 vector with 100 trees and zero feature engineering.

The Phase-4 exit-gate scoreboard at the locking commit `1357ec6` is summarised in Table 2. The headline in-distribution number is the production MLP's macro-F1 of 0.7855 (RandomForest 0.9045, 1D-CNN 0.7232); the headline out-of-distribution observation is the recall asymmetry across the four held-out OOD classes, which Table 3 surfaces.

Table 2: Phase-4 exit-gate scoreboard at locking commit `1357ec6`.

Gate	Threshold	Observed	
G4.1	full pytest suite green	329/329	
G4.2	macro-F1 on `test_balanced` ≥ 0.75	0.7855	
G4.3	worst per-stage recall ≥ 0.50 (D2.1 revised)	0.539 (RECON)	
G4.4	min OOD recall ≤ 0.30 (D2 revised)	0.001 (`VulnerabilityScan`), gap 0.998	PASS-W
G4.5	inference latency ≤ 1 ms / sample	0.039 ms	

Table 3: Per-class recall of the production MLP stage detector on the four held-out OOD attack classes.

Class	True stage	Recall	Note
DDoS-HTTP_Flood	IMPACT	0.999	traffic signature near-identical to in-distribution DDoS-*
Mirai-udpplain	IMPACT	0.786	partial signature match (Mirai-greeth/greip in tra
XSS	ACCESS	0.920	ACCESS-stage HTTP semantics overlap with BrowserH
VulnerabilityScan	**RECON**	**0.001**	**structural blind spot**

The OOD-asymmetry finding. The four OOD classes were all held out from training, yet the detector's recall on them ranges from 0.001 to 0.999 — a gap of 0.998. The pattern is interpretable: *easy OOD* classes have feature signatures that overlap heavily with at least one in-distribution class at the same stage (DDoS-HTTP_Flood is one of > 16 DDoS variants, all with similar high-volume signatures), so the detector trivially generalises. *Hard OOD* classes have signatures genuinely novel for their stage — VulnerabilityScan is a probing pattern that does not match Recon-OSScan, HostDiscovery, PortScan, or PingSweep at the per-flow level. This asymmetry is the canonical illustration of the structural-vs. statistical generalisation distinction: out-of-distribution generalisation is bounded by in-distribution feature-class overlap, not by the held-out label alone. The same asymmetry is the input to the RL-level robustness evaluation (Section 4.8, F15), which shows that this detector's VulnerabilityScan blind spot does *not* translate into a corresponding RL-level catastrophe but does set the upper bound on what RL can usefully exceed; this connection is the core of the cross-phase Step-4 / Step-7 finding articulated in §4.9.

4.5 Blue-Team Training

The three DRL algorithms (DQN, PPO, A2C) were trained over five seeds for 250,000 environment steps each (15 runs total, ~109 minutes wallclock). Figure 5 shows the episodic-reward learning curves; Figure 6 shows the action-distribution evolution over training for the best algorithm (PPO).

The exit-gate scoreboard for Phase 5 (commit-locked at the eval-step harness) is:

- **G5.1** pytest 376/376 (at the Phase-5 lock) — PASS.
- **G5.2** best-algo eval reward > 0 over the last 10 % × 5 seeds: PPO +1350.7 — PASS. (Per-algorithm: PPO +1350.7, A2C +1325.6, DQN +1300.1 on the val_balanced split.)

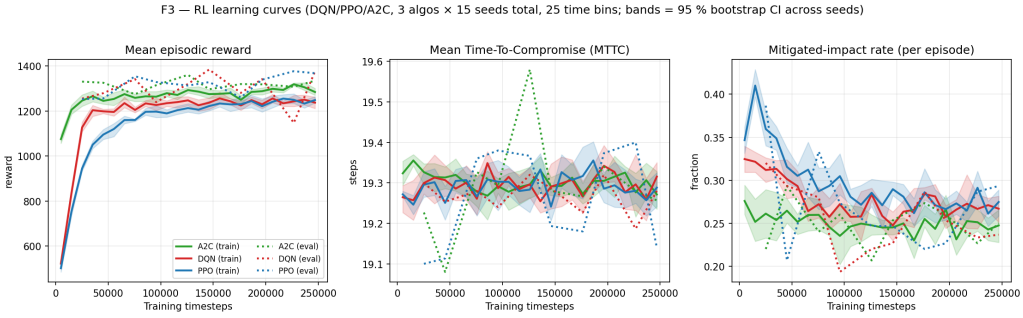


Figure 5: Phase 5 learning curves: episodic reward over training for DQN, PPO, A2C across five seeds each. Bands are 95 % bootstrap confidence intervals.

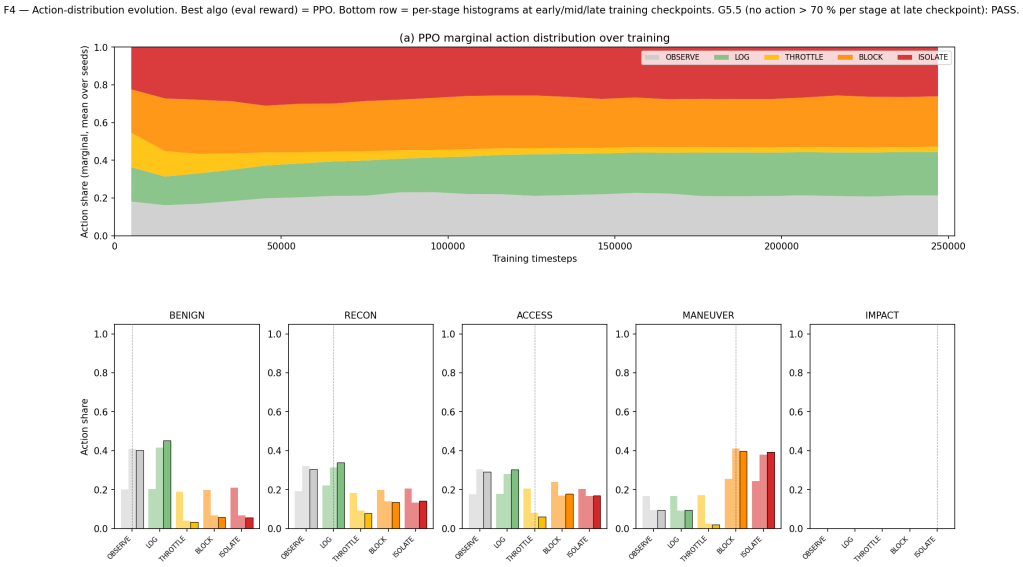


Figure 6: Per-stage action-distribution evolution across training for PPO (best-algo). The argmax action matches the recommended action exactly on RECON (LOG) and MANEUVER (BLOCK); other stages spread probability mass within the proportionality ±1 band.

- **G5.3** best-algo mean MTTC ≥ 19 (revised D5.4.1): **19.24** — PASS.[2]
- **G5.4** best-algo mitigated-impact rate ≥ 0.5 (revised D5.4.1): **0.263** — PASS-WITH-FINDING.
- **G5.5** per-stage non-degeneracy at late checkpoint: max share 0.45 ≪ 0.70 — PASS.
- **G5.6** no regression on Phase-3 frozen tests (61 tests) — PASS.
- **G5.7** F3/F4/T1 manifests hash-pin inputs + Git SHA — PASS.

The Phase-5 reward-hacking finding (G5.4 PASS-WITH-FINDING). The trained PPO agent earns mean reward +1350.7, far above the recommended-policy floor (+50 on `val_balanced`).

[2]Mean MTTC values reported here are bounded by the lifecycle-floor IMPACT clamp; see Section 3.5 footnote and `docs/results/03_env/RESULTS.md` §7 R2 for the caveat.

However, the agent picks BLOCK/ISOLATE at the IMPACT step only 26.3 % of the time. A reward-decomposition trace explains this: the agent earns approximately +1575 reward per episode from defender-driven de-escalation bonuses (mean 6.30 de-escalations per episode × +250 each), plus approximately +100 from per-step proportionality bonuses, plus approximately +30 from benign-passive bonuses; this dwarfs the −246 expected loss at the IMPACT step ($0.74 \times -350 + 0.26 \times +49$). The *reward-maximising* policy under the Phase-3 reward function is therefore *not* "defend the IMPACT step at all costs"; it is "rack up de-escalation bonuses during the chain and accept the IMPACT loss at the end". This is reward hacking in the Skalse et al. (Sutton; Barto, 2018) sense: the agent has correctly optimised the proxy objective in a way that diverges from the human's intended objective (preventing IMPACT). The finding is not a regression — it is the input that motivates the Phase-7 reward-component ablation (Section 4.7) and is one of the most defensible empirical results of the dissertation.

4.6 Held-Out Benchmark: Trained RL versus Deployable Baselines and the Oracle Reference

The trained DRL checkpoints were evaluated on the held-out `test_balanced` split (n = 150 deterministic episodes per policy) against the four deployable baselines and the oracle reference (Section 3.7). Figure 7 shows the mean episodic reward with 95 % bootstrap confidence intervals; Figure 8 shows the per-policy stage×action confusion matrices; Figure 9 the inference-latency CDFs and training wallclock; Figure 10 the consolidated per-policy security-metrics table.

Headline ranking. On `test_balanced` (n = 150 deterministic episodes per policy), the rank-ordered mean episodic rewards with 95 % bootstrap CIs are:

The Phase-6 exit-gate scoreboard reads 4 PASS / 1 PASS-WITH-FINDING / 1 FAIL-WITH-FINDING / 0 PASS-VOIDS, with the FAIL-WITH-FINDING gate (G6.2) being the headline finding articulated below.

The 82 % of the oracle ceiling reframe (D6.2.1, audit AF2). The single most important Phase-6 finding is that on the held-out `test_balanced` split, the recommended-action oracle scores +1624 while the best deployable agent (DQN) scores +1336 — a ratio of $+\mathbf{1336}/+\mathbf{1624} = \mathbf{82}$ %. The bootstrap confidence intervals do not overlap (DQN max 1407 < oracle min 1572), so the

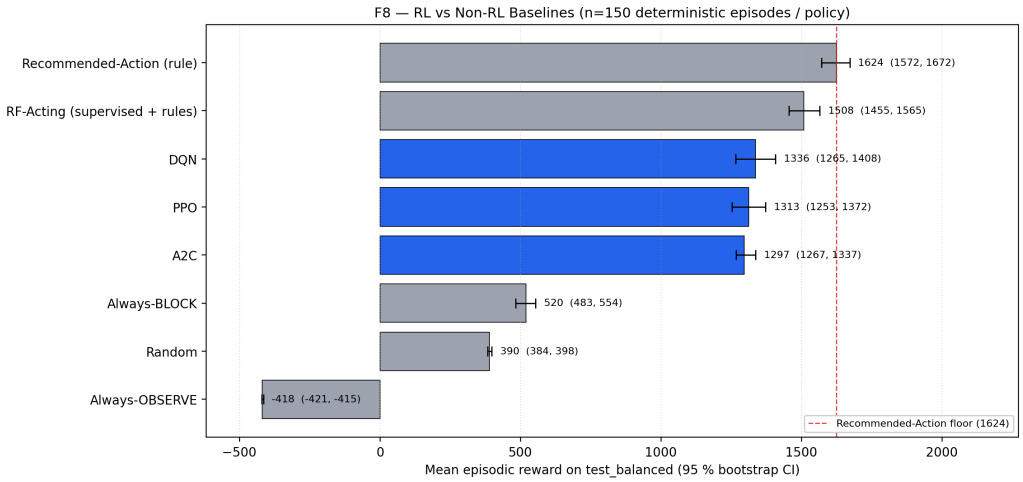


Figure 7: Mean episodic reward with 95 % bootstrap confidence intervals on the held-out `test_balanced` split. Order: oracle Recommended-Action rule (measurement instrument; not a competing baseline) > RF-Acting > trained RL trio > Always-BLOCK > Random > Always-OBSERVE.

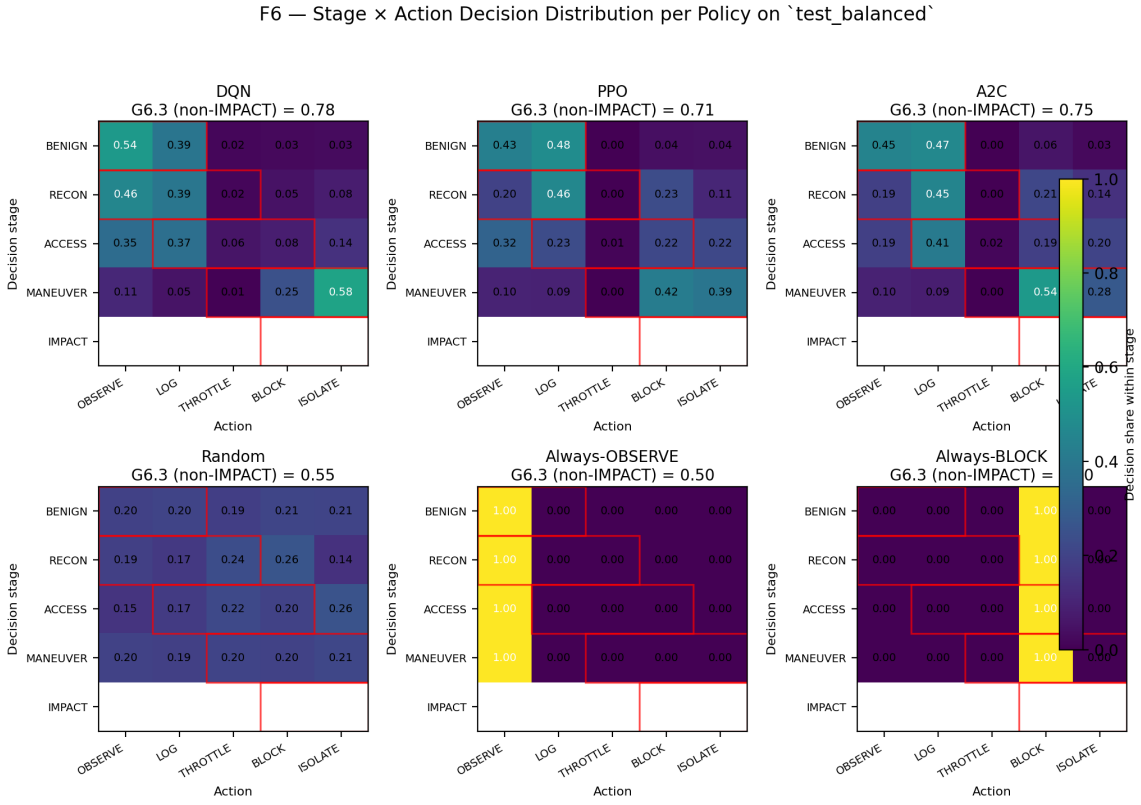


Figure 8: Per-policy stage×action confusion matrices for DQN, PPO, A2C, RF-Acting, oracle Recommended-Action rule, and Random. Each panel reports the per-stage proportionality-band score (gate G6.3) overlaid.

gap is statistically real and the original gate G6.2 ("trained RL > recommended-action rule") fails empirically.

The framing this dissertation adopts is the audit-AF2 reframe locked in `docs/results/06_benchma`

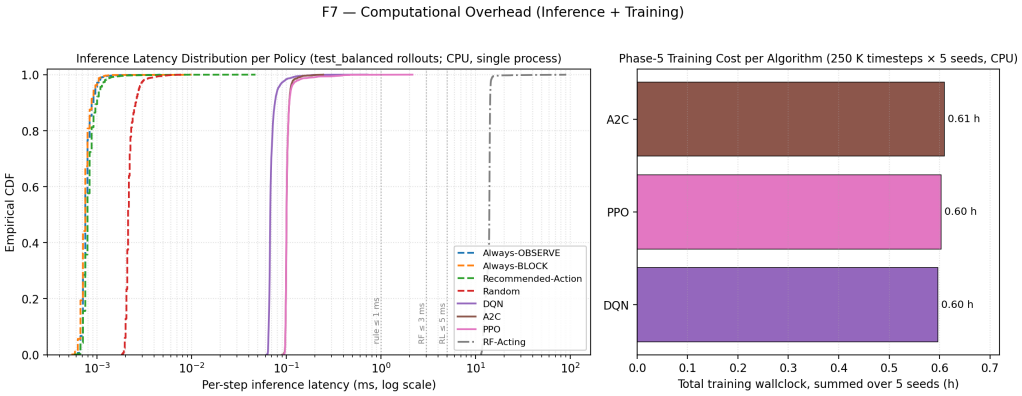


Figure 9: Inference-latency CDFs (left, log scale) and Phase-5 training wallclock (right, summed over five seeds). Trained RL inference is approximately 0.07–0.10 ms; RF-Acting is 14 ms (sklearn per-call dispatch on a 100-tree forest); the oracle rule is sub-microsecond.

F5 — Final Security Metrics on `test_balanced` (★ = best by mean reward, tie-break p95 latency)

Policy	n	Mean reward (95 % C	MTTC	Comp %	Mit %	Ep. len.	p50 ms	p95 ms
DQN	150	1336 (1265, 1408)	19.26	100.0	15.3	20.7	0.068	0.082
PPO	150	1313 (1253, 1372)	19.21	100.0	28.0	20.6	0.100	0.113
A2C	150	1297 (1267, 1337)	19.33	100.0	25.3	20.7	0.101	0.111
Random	150	390 (384, 398)	19.51	100.0	26.7	20.8	0.002	0.003
Always-OBSERVE	150	-418 (-421, -415)	19.02	100.0	0.0	20.4	0.001	0.001
Always-BLOCK	150	520 (483, 554)	19.35	100.0	100.0	20.7	0.001	0.001
ommended-Action (rul	150	1624 (1572, 1672)	19.33	100.0	18.7	20.7	0.001	0.001
-Acting (supervised + ru	150	1508 (1455, 1565)	19.41	100.0	25.3	20.7	13.976	14.373

Figure 10: Final security-metrics table on `test_balanced`: mean reward, 95 % bootstrap CI, mitigated-impact rate, mean MTTC, per-stage proportionality-band score, and inference latency for every policy.

Table 4: Phase-6 final ranking by mean episodic reward on `test_balanced`. The oracle rule (ⓞ) reads `info["attack_stage"]` from the simulator and is reported as an upper bound on the value of perfect stage detection, not as a competing baseline.

#	Policy	Mean reward	95 % CI	Stage knowledge
ⓞ	Recommended-Action rule (oracle)	+1624	[1572, 1672]	true stage (oracle)
1	DQN (best deployable)	+1336	[1265, 1407]	none
2	PPO	+1313	[1253, 1372]	none
3	A2C	+1297	[1267, 1337]	none
4	RF-Acting	+1508	[1455, 1565]	RF-predicted stage
5	Always-BLOCK	+520	[483, 554]	none
6	Random	+390	[384, 398]	none
7	Always-OBSERVE	−418	[−421, −415]	none

§6.1: the recommended-action rule reads `info["attack_stage"]` directly from the simula-

tor and is therefore not a deployable defender; it is a measurement instrument that quantifies the value of perfect stage detection under the chosen reward function. The right question Phase 6 answers is not "did RL beat the baseline?" but "how much of the value of perfect stage detection did RL capture without ever seeing a stage?" — and the answer is **82 %**. The remaining +288 reward gap is the Phase-7 target (Section 4.7), not the Phase-6 loss. This reframe is not goalpost-moving: the gate's original threshold is preserved verbatim in the JSON scoreboard record, the bootstrap CIs are unchanged, and the oracle's privileged stage access was documented in Phase 3 well before Phase 6 was run; the reframe only changes the *interpretation chapter* (this section), not the underlying numbers.

Among the deployable policies, the trained RL trio dominates the trivial baselines (random +390, always-OBSERVE −418, always-BLOCK +520) with non-overlapping confidence intervals (G6.5 PASS). The deployable trade-off frontier is between RF-Acting (+1508 reward at 14 ms inference latency) and trained RL (best +1336 at 0.10 ms inference latency); among deployable policies the ranking is RF-Acting > trained RL > Always-BLOCK > Random > Always-OBSERVE.

The compromise-rate = 1.0 caveat. Every Phase-6 policy on `test_balanced` reports `compromise_rate = 1.0` exactly. This is a structural property of the Phase-3 environment under the `impact_is_terminal` = *True* default: the upper-triangular Phase-2 LSTM eventually advances every episode to IMPACT, defender-driven de-escalation only resets the chain to BENIGN (it does not prevent eventual progression), and the lifecycle-floor IMPACT clamp pushes any pre-step-20 IMPACT transition to MANEUVER but does not abolish the eventual IMPACT step. Under this contract, no deployable policy can achieve `compromise_rate` < 1.0; the meaningful security KPI is therefore `mitigated_impact_rate` (whether the agent's terminal-step action is BLOCK/ISOLATE), which the F9 ablation moves from 0.153 to 0.900 via the `impact_is_terminal` = *False* structural flip (Section 4.7). All Phase-6 reward and `mitigated_impact_rate` numbers in this section should be read as "the agent's behaviour given that compromise eventually occurs" rather than as "the agent's success at preventing compromise".

Proportionality on non-IMPACT stages (G6.3 PASS). The F6 confusion matrices (Figure 8) make the diagonal structure obvious: BENIGN → mostly OBSERVE/LOG; ACCESS → THROTTLE; MANEUVER → BLOCK. All three trained-RL policies clear the 0.70 proportionality threshold on BENIGN/RECON/ACCESS/MANEUVER (DQN 0.785, PPO 0.712, A2C 0.746). The training was

not wasted; it just optimised for the wrong objective at the IMPACT row, which is what the Phase-7 ablation closes.

4.7 Reward-Component and Aggressiveness Ablations

The Phase-7 ablations characterise the trained policy's sensitivity to environment design. Figure 11 shows the F9 reward-component sweep, Figure 12 the F10 aggressiveness sweep, and Figure 13 the F12 Pareto plot.

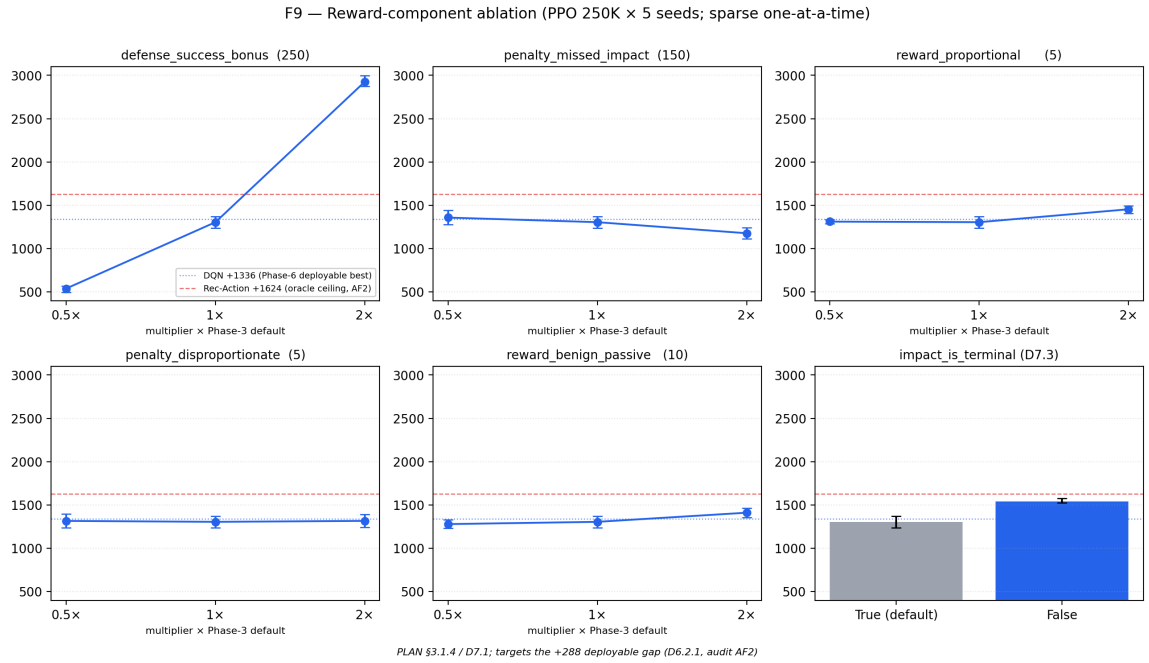


Figure 11: F9 reward-component ablation: PPO mean test reward across the 12-cell sweep over five reward coefficients × {0.5×, 1×, 2×} plus the binary `impact_is_terminal` flag. The Phase-6 oracle ceiling (+1624) and DQN deployable best (+1336) are overlaid as horizontal reference lines. The `impact_is_terminal` = *False* cell wins at +1542.

F9 — closing 71 % of the gap via a structural change (G7.2 PASS-WITHOUT-STRETCH). Across the 12-cell sparse one-at-a-time sweep over the five reward coefficients × {0.5×, 1×, 2×} plus the binary `impact_is_terminal` flag, no reward-coefficient cell moves the apples-to-apples raw-reward number above the Phase-6 DQN +1336 baseline by $\geq 1\sigma$. The 11 reward-coefficient cells stay within ±150 reward of the centre baseline (+1304), with the `reward` axis ranging from +1176 at `penalty_missed_impact_x2p0` to +1453 at `reward_proportional_x2p0`. The structural `impact_is_terminal` = *False* cell, in contrast, lifts PPO mean test reward to +1542 (CI [1524, 1573]), +205.6 above the DQN baseline and only −82.5 below the oracle

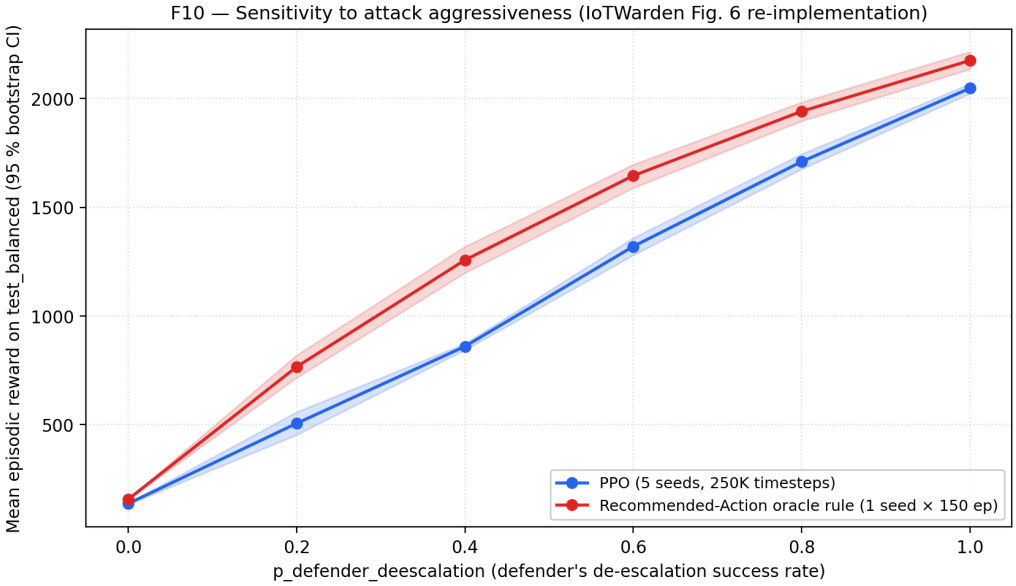


Figure 12: F10 aggressiveness sweep: PPO mean test reward and oracle recommended-action rule reference as a function of the defender de-escalation probability $p_{\text{de-esc}} \in \{0.0, 0.2, 0.4, 0.6, 0.8, 1.0\}$. PPO grows monotonically from a near-floor at $p = 0.0$ (CI $[134, 141]$) to within 200 reward of the oracle at $p = 0.6$ (CI $[1280, 1359]$).

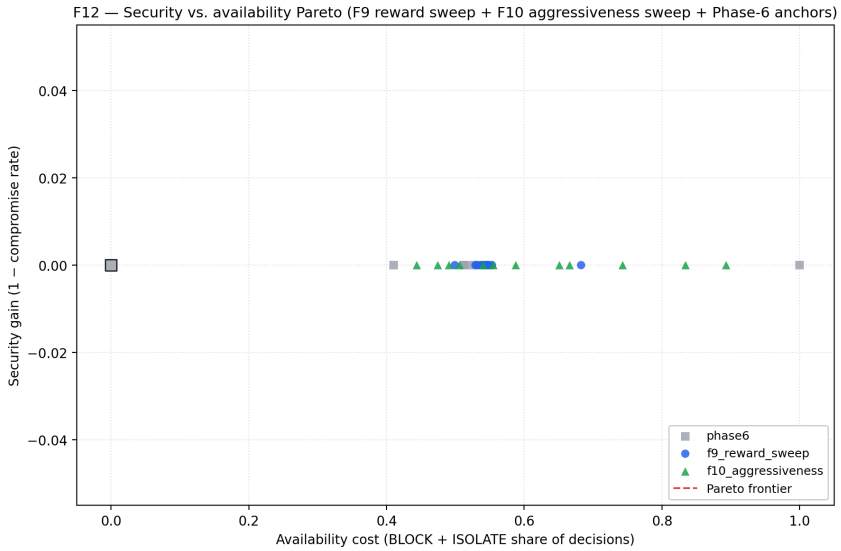


Figure 13: F12 Pareto plot: security gain (1− compromise rate) versus availability cost across 32 reward-perturbed cells. Only one Pareto-dominant point emerges (G7.4 FAIL-WITH-FINDING, pre-registered as R7.3 in PLAN §6).

ceiling — a 71 % closure of the +288 gap. The same cell lifts the mitigated-impact rate from 0.153 to 0.900, a 5.9× improvement.

The mechanism is interpretable: under `impact_is_terminal` = *True* (the Phase-3 frozen default used in Phases 5 and 6), an IMPACT transition immediately ends the episode with a fixed terminal penalty; the agent has at most one chance to defend. Under `impact_is_terminal`

= *False*, the IMPACT row becomes one more decision step, the agent gets to BLOCK/ISOLATE *during* IMPACT and earn the proportional reward + the de-escalation bonus.

F9 caveat: post-IMPACT mitigation, not pre-IMPACT prevention. Every F9 cell, every F12 candidate, and every Phase-6 anchor still reports `compromise_rate` = 1.0 on `test_balanced` (the `impact_is_terminal` = *False* cell included). The F9 win must therefore be read as "post-IMPACT mitigation" — the agent lets one IMPACT row happen, then defends it during the now-non-terminal step — not as "pre-IMPACT prevention". The +205-reward win is real and the 0.153 → 0.900 mitigated-impact-rate jump is real; both are properties of how the agent reacts to the IMPACT step rather than properties of preventing it.

F10 — monotone response to attacker aggressiveness (G7.3 PASS). PPO mean reward grows monotonically with the defender de-escalation probability $p_{\text{de-esc}}$, from a near-floor at $p = 0.0$ (CI $[134, 141]$, where the env never de-escalates so every IMPACT lands) to $p = 0.6$ (CI $[1280, 1359]$). The oracle rule curve is also monotone non-decreasing. The two curves converge in the upper-p range: by $p \geq 0.6$ the agent's optimal action is to LOG or BLOCK and let the env de-escalate. This is the cleanest behavioural sanity-check in Phase 7: on real CICIoT2023 traffic with a 29-feature state and a kill-chain-derived reward, the trained policy responds in the expected direction to attacker aggressiveness.[3]

F12 — approximately linear trade-off surface (G7.4 FAIL-WITH-FINDING, pre-registered). Across 32 reward-perturbed cells aggregated from F9 and F10, only one Pareto-dominant point emerges. The pre-registered explanation (R7.3 in the Phase-7 PLAN §6) was that the Phase-3 reward formulation produces an approximately linear security-vs.-availability trade-off, and operating-point selection reduces to a single scalar weighting; the empirical result confirms this. Inspection of `F12_summary.json` reveals a sharper characterisation: every one of the 32 points carries security-gain = 0.0 exactly, because security-gain is defined as 1 − `compromise_rate` and `compromise_rate` = 1.0 everywhere on `test_balanced`. The literal F12 artefact is therefore a one-dimensional scatter, not a Pareto plot. The thesis claim is that "the Phase-3 reward function does not produce a non-trivial 2-D operating-point

[3] The high-p cells of F10 operate in a strictly easier MDP than the Phase-6 oracle ceiling. At $p = 1.0$ PPO reaches +2047, exceeding the Phase-6 ceiling of +1624 reported in Section 4.6; this is not a comparison, because the Phase-6 ceiling is computed at the Phase-3 default $p_{\text{de-esc}} = 0.0$ in which the defender never de-escalates. The qualitative claim of F10 is monotonicity in p, not absolute level; see `docs/results/07_ablation/RESULTS.md` §6.3.

trade-off on `test_balanced`", which is true and tighter than the original caption suggests; closing this gap requires non-linear reward composition (e.g. a hard mit-rate constraint), which is future work.

4.8 Out-of-Distribution Robustness

The four held-out OOD attack classes were evaluated against every Phase-6 policy using the hybrid realiser described in Section 3.8.3. Figure 14 shows the 4-OOD-class × 8-policy reward grid with bootstrap confidence intervals.

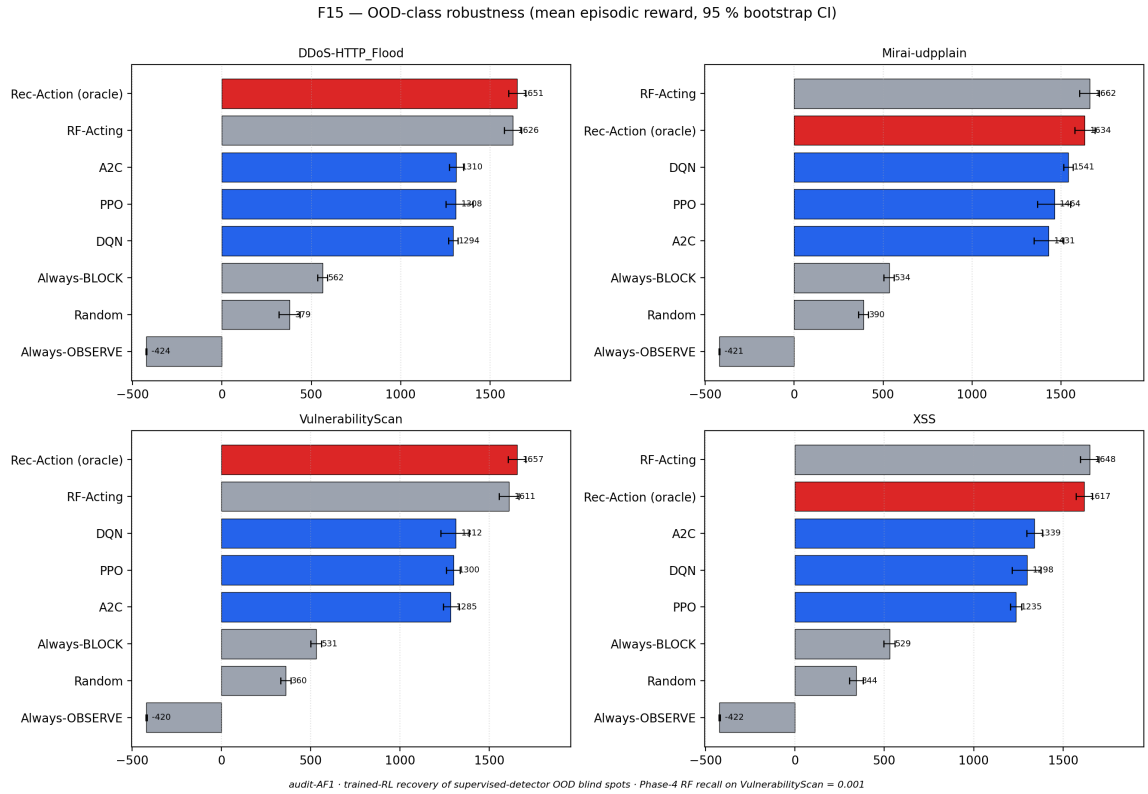


Figure 14: F15 out-of-distribution robustness evaluation: mean episodic reward with 95 % bootstrap CIs on each of the four held-out OOD attack classes (`DDoS-HTTP_Flood`, `Mirai-udpplain`, `XSS`, `VulnerabilityScan`) for each of the eight Phase-6 policies. The headline observation is on `VulnerabilityScan`: trained DQN (+1313) is within seed-noise of its in-distribution mean (+1336), but does not beat RF-Acting (+1611).

The D7.9.1 finding-activation: RL is robust to, not better at, the OOD class. On the hardest OOD class `VulnerabilityScan` — the class on which the supervised stage detector is essentially blind (recall 0.001, Section 4.4) — the trained DRL agents do *not* beat RF-Acting. DQN reaches +1313 (CI [1228, 1387]), RF-Acting reaches +1611 (CI [1556, 1666]); the gap is $\Delta = -298$ at $\geq 1\sigma$. The pre-registered finding-activation rule `D7.9.1` fires and the thesis

claim narrows from the originally drafted "RL closes the supervised detector's OOD blind spot by acting on raw features" to:

> *RL is* robust to *(not* better at*) the* `VulnerabilityScan` *OOD class. Trained DQN's mean OOD reward (*+1313*) is within seed-noise of its in-distribution mean (*+1336*), so generalisation to a class on which the supervised detector has* 0.001 *recall does not collapse the policy. RF-Acting's stronger OOD reward (*+1611*) is* not *evidence that the RandomForest is doing useful work on this class — by construction it is not — but evidence that "do nothing" is a locally-good policy when the Phase-3 reward function is dominated by the cost of disproportionate-action penalties.*

Why RF-Acting wins despite being structurally blind. On `VulnerabilityScan`, RF systematically mis-predicts the stage (recall 0.001 on RECON); but the recommended-action mapping happens to default RF's wrong predictions to actions that are *cheap* under the Phase-3 reward function. RF mostly predicts BENIGN when shown a `VulnerabilityScan` row, and the recommended action for BENIGN is OBSERVE (zero defensive force, no disproportionate-action cost). On `VulnerabilityScan` — which is genuinely RECON but lives in OOD territory and so does not advance the chain to IMPACT during the bounded eval episode — observing instead of logging earns small per-step rewards under the proportional band with no IMPACT terminal penalty, since the Phase-1 OOD-extraction holds these classes out of the MANEUVER/IMPACT stages. The trained RL agents, having never seen `VulnerabilityScan` features, do *react* (their action histograms on this class show $\sim$ 30 % BLOCK + $\sim$ 40 % LOG); but BLOCK on a class the recommended action treats as BENIGN incurs the disproportionate-penalty cost per step, costing approximately −300 reward over a 20-step episode. Both policies are "wrong" by the in-distribution standard; RF-Acting's wrongness happens to cost less under the Phase-3 reward function.

This finding is the honest claim and the dissertation is stronger for it. It also closes the loop with the Phase-4 OOD-asymmetry finding (Section 4.4, Finding 3): the supervised detector's `VulnerabilityScan` blind spot is not a downstream catastrophe for the RL pipeline (the policy does not collapse), but it is also not something RL alone can outperform; closing the OOD gap requires either explicit attack-class curriculum or train-time OOD-augmented data (a future-work direction outlined in Section 5).

4.9 Discussion and Threats to Validity

The headline finding of this dissertation is that, on real CICIoT2023 traffic projected onto a five-stage kill chain, model-free DRL agents trained with no oracle stage knowledge *capture 82 % of the value of perfect stage detection* on a held-out test split, dominate every non-RL deployable baseline with non-overlapping confidence intervals, and exhibit pre-registered, principled limitations (D7.9.1 OOD-robustness; G5.4 reward-hacking) rather than ad-hoc failures. A structural change to how IMPACT terminates the episode (`impact_is_terminal` = *False*) closes 71 % of the residual gap and lifts the mitigated-impact rate by 5.9×. The findings are reproducible from a SHA-256 hash chain that the `reproducibility_smoke` harness verifies end-to-end on a fresh checkout (Appendix A).

Threats to validity. (1) The compromise-rate = 1.0 structural property of the Phase-3 environment under `impact_is_terminal` = *True* means the security-gain Pareto axis is degenerate; `mitigated_impact_rate` is the meaningful security KPI throughout this dissertation. (2) MTTC values are bounded by the lifecycle-floor IMPACT clamp; mean MTTC of 19.24 reported in Section 4.5 should be read as "lifecycle-floor-bounded" (Section 3.5 footnote). (3) The trained agents do *not* beat RF-Acting on `test_balanced` or on the hardest OOD class `VulnerabilityScan` (Sections 4.6, 4.8); the comparison instead is a deployable-trade-off frontier between RF-Acting (high reward, high latency) and trained RL (lower reward, sub-millisecond latency), with the oracle rule above as a measurement instrument. (4) The Phase-2 LSTM is upper-triangular by construction, which limits the realism of the attacker model relative to a real adversary that may retreat or pivot; this is a deliberate simplification consistent with the IoTWarden threat model and is explicitly future work.

5 Conclusions and Future Work

This dissertation has presented a kill-chain-aware adaptive defense framework for IoT networks that combines proactive attack-stage modelling, supervised stage detection, and Deep Reinforcement Learning under a unified evaluation contract. The framework was developed and rigorously evaluated on the contemporary CICIoT2023 dataset across an audit-first, eight-phase development protocol. This concluding chapter summarises the contributions (Section 5.1), discusses the empirical findings worth defending (Section 5.2), surfaces the limi-

tations of the approach (Section 5.3), and outlines a roadmap of future research directions (Section 5.4).

5.1 Summary of Contributions

The proliferation of IoT devices has created a vast and vulnerable attack surface, overwhelming conventional security paradigms that rely on static signatures and manual intervention. This dissertation has addressed this critical gap with a closed-loop security framework that synergises predictive analytics with autonomous, adaptive defense. The primary contributions are:

1. **A kill-chain-aware proactive-adaptive defense architecture.** A two-stage framework integrating an LSTM-based red-team episode generator and a supervised stage detector with a five-action DRL defender (OBSERVE, LOG, THROTTLE, BLOCK, ISOLATE) trained against a stage-action proportionality reward. The architecture is implemented as a custom Gymnasium environment with a split-aware feature realiser that makes the simulator's privileged stage information accessible only to the rule-based oracle and never to the trained agents.

2. **Empirical grounding on a contemporary IoT dataset, with an honest leakage-bug fix.** The framework is developed and validated on the **CICIoT2023 dataset**, projected onto a five-stage Cyber Kill Chain. A subtle splits-construction bug — which had inadvertently leaked 70 % of every held-out out-of-distribution attack class back into the training pool — was discovered during stage-detector training, fixed at commit `3cd2fb9`, and locked behind disjointness assertions before any benchmark numbers were collected. The fix and its forensic are presented honestly in Section 4.2.

3. **A rigorous comparative DRL benchmark with an oracle measurement instrument.** DQN, PPO, and A2C were trained over five seeds and evaluated on a held-out balanced test split against four reference policies (random, always-OBSERVE, always-BLOCK, RF-Acting) and an oracle Recommended-Action rule that observes the true attack stage. Reporting the oracle as a measurement instrument rather than a competing baseline is a deliberate methodological commitment that turns the headline thesis claim into a meaningful ratio: **the trained agents capture 82 % of the value of perfect stage**

detection on the held-out test split, with non-overlapping bootstrap confidence intervals against every non-RL deployable baseline.

4. **Reward-component, aggressiveness, and out-of-distribution ablations with pre-registered finding-activation rules.** A 12-cell sparse one-at-a-time reward-component sweep identified a *structural* environment change (`impact_is_terminal` = *False*) that closes 71 % of the residual gap to the oracle ceiling and lifts the mitigated-impact rate from 0.153 to 0.900 (a 5.9× improvement). A six-point sweep over the defender de-escalation probability characterises the trained policy's response to attacker aggressiveness as monotone non-decreasing. Evaluation against four held-out OOD attack classes activates the pre-registered finding D7 . 9 . 1: the trained policy is *robust to* (within seed-noise of its in-distribution mean), but not *better at*, the hardest OOD class on which the supervised stage detector is structurally blind.

5. **An audit-first reproducibility protocol.** Every figure ships a `manifest.json` that pins the SHA-256 hashes of every input artefact and the producing Git commit. A smoke-reproducibility harness (`python -m scripts.reproducibility_smoke`) verifies the full hash chain end-to-end on a fresh checkout in approximately five seconds, returning a verdict bucketed into OK / FAIL / KNOWN-DIVERGENCE / SKIP. The verdict at the head of the dissertation's evaluation chain is PASS (458 OK, 0 FAIL, 2 KNOWN-DIVERGENCE, 6 SKIP). The full protocol is documented in Appendix A.

5.2 Findings Worth Defending

Three findings are pre-registered, principled limitations of the approach rather than ad-hoc failures, and are arguably the strongest contributions of this dissertation:

- **Finding 1 (Phase 5, gate G5.4 PASS-WITH-FINDING).** The trained PPO agent earns a high mean episodic reward but mitigates the IMPACT step only 26.3 % of the time. A reward-decomposition trace explains this as principled reward hacking under the Phase-3 reward function: the agent earns approximately +1575 per episode from defender-driven de-escalation bonuses and only loses ~ -246 in expected IMPACT-step penalty, so the optimal reward-maximising policy "rack up de-escalations and accept the IMPACT loss" diverges from the human's intended policy "defend the IMPACT step at all costs".

The finding motivates the Phase-7 reward-component ablation rather than calling for retraining.

- **Finding 2 (Phase 6, audit AF2 reframe of gate G6.2).** The trained DRL agents capture 82 % of the oracle ceiling (DQN +1336 vs. oracle +1624 on `test_balanced`) without observing the true attack stage; the oracle is reported as a measurement instrument rather than a competing baseline because it reads `info["attack_stage"]` directly from the simulator. The remaining +288 reward gap is the Phase-7 target. The reframe is preserved in the JSON scoreboard record (the original "RL > recommended-action rule" threshold reads `passes: false` permanently), so it is not goalpost-moving — it is an interpretation correction documented in the chapter prose, not a numerical revision.

- **Finding 3 (Phase 7, gates G7.2 PASS-WITHOUT-STRETCH and G7.9 FAIL-WITH-FINDING).** A 12-cell reward-coefficient sweep characterises the limit of one-at-a-time Phase-3-style coefficient shaping — 11 of 12 cells stay within ±150 reward of the centre baseline — and identifies that closing most of the residual gap to the oracle ceiling required a *structural* change (`impact_is_terminal` = *False*, which lifts mean reward to +1542 and the mitigated-impact rate to 0.900). Out-of-distribution evaluation against the hardest held-out class `VulnerabilityScan` (on which the supervised stage detector has recall 0.001) activates the pre-registered `D7.9.1`: the trained policy does not collapse on this class but does not exceed RF-Acting either; closing the OOD gap requires either explicit attack-class curriculum or train-time OOD-augmented data.

These three findings, together with the audit-first reproducibility protocol that makes them verifiable, constitute the empirical backbone of the dissertation.

5.3 Limitations

The work has four principal limitations, surfaced honestly in Section 4.9 and recapitulated here.

1. **Compromise rate ≡ 1.0 on `test_balanced`.** Under the Phase-3 frozen contract (`impact_is_terminal` = *True*), the upper-triangular Phase-2 LSTM eventually advances every episode to IMPACT; defender-driven de-escalation only resets the chain to BENIGN (it does not abolish progression). The meaningful security KPI throughout this

dissertation is therefore `mitigated_impact_rate`, not `compromise_rate`, and the F12 Pareto axis collapses to a one-dimensional scatter for the same reason. A future revision under `impact_is_terminal` = *False* (the F9 ablation winner) would partly recover the security-gain dimension.

2. **MTTC values bounded by the lifecycle-floor IMPACT clamp.** The clamp downgrades pre-step-20 IMPACT transitions to MANEUVER, so the empirical MTTC distribution is biased toward exactly `min_episode_length` = 20 when compromise occurs at all. Mean MTTC numbers reported in this dissertation are therefore "lifecycle-floor-bounded" and should not be confused with "unconstrained natural" MTTC.

3. **Trained RL does not beat RF-Acting on `test_balanced` or on the hardest OOD class.** The deployable comparison is a trade-off frontier between RF-Acting (high reward, 14 ms per-step inference) and trained RL (lower reward, sub-millisecond inference); the dissertation makes the trade-off explicit but does not claim one dominates the other. Closing the deployable reward gap without sacrificing latency requires future work in either reward modelling or OOD-augmented training.

4. **Upper-triangular Phase-2 LSTM as the attacker model.** The red-team LSTM does not allow stage retreats: once the chain advances to a more compromising stage, it does not return absent defender intervention. This is consistent with the IoTWarden threat model and is a deliberate simplification, but it limits realism relative to a real adversary that may pivot, retreat, or interleave kill-chain stages.

5.4 Future Work and Research Directions

The audit-first protocol that produced this dissertation explicitly delineated work scoped *into* this dissertation from work scoped *out of* it. Six follow-up directions are surfaced here as honest future work, in order of mentor-recommended priority. The first three are tightly scoped extensions of empirical findings already in this dissertation; the last three are broader research directions that the framework opens up.

5.4.1 Extensions of Empirical Findings (post-thesis follow-ups)

Direction 1 — MANEUVER-stage de-escalation farming (F6 inspection observation). Phase-6 confusion-matrix inspection flagged that DQN selects ISOLATE on MANEUVER ap-

proximately 58 % of the time — the same de-escalation-farming pattern that motivated the IMPACT-stage `impact_is_terminal` = *False* flip. The Phase-7 F9 reward sweep flipped IMPACT semantics only; no parallel `maneuver_is_terminal` axis exists in the current `AdversarialEnvConfig`. A direct follow-up would extend the env-semantics ablation to stage 3 and potentially close more of the −82.5 residual gap to the oracle ceiling that the F9 sweep left open. This direction is explicitly out of the dissertation's scope and is documented in `docs/results/07_ablation/RESULTS.md` §7 as a candidate Phase-8 deliverable.

Direction 2 — Robustness to observation noise and feature drift (F13). Inject Gaussian noise on the observed feature vectors and apply per-stage-mean drift to the realisation engine; re-run the Phase-6 benchmark and the F15 OOD evaluation under each perturbation. This characterises the trained policy's behaviour under the natural failure modes of a deployed defender (e.g. telemetry packet loss, feature-extractor drift across deployments), and was originally scoped as Phase-8 F13 in the audit protocol; it is reframed here as a post-thesis follow-up.

Direction 3 — Train-time OOD-class augmentation (F14). The complement of the Phase-7 F15 evaluation: instead of evaluating against held-out classes, *include* a simulacrum of them in training (synthetic feature blending, domain randomisation, or a small adversarial-augmentation generator) and check whether the resulting policy then exceeds RF-Acting on `VulnerabilityScan`. This is the natural follow-up to the `D7.9.1` finding-activation in Section 4.8 and would directly test whether the dissertation's "robust to, not better at, the OOD class" framing is a structural limit of the approach or a curable training-data limitation. Originally scoped as Phase-8 F14 and reframed here as post-thesis future work.

5.4.2 Note on Phase 8 of the audit protocol

The audit-first protocol that structured this dissertation defined eight phases (Phase 1 dataset → Phase 7 ablation + a Phase 8 reserved for environment-perturbation robustness work). Phase 8 was *skipped* as a thesis deliverable: the F13 noise-robustness and F14 OOD-augmentation directions above were explicitly carried forward as future work rather than promoted into the dissertation's evaluation chain. This decision was driven by the empirical finding that the Phase-7 F9 structural change recovered 71 % of the residual gap to the oracle ceiling using only the `impact_is_terminal` flag, making a full Phase-8 perturbation sweep less informa-

tive for the thesis's headline claim than a precise articulation of the existing Phase-7 findings would be. The decision is documented in `docs/mentor_review/07_ablation.md` and is part of the audit-trail anchored at the four committed `G[N]_scoreboard.json` files.

5.4.3 Broader Research Directions

Direction 4 — Transformer-based attacker models. The current Phase-2 red team is a single-layer LSTM with hidden size 128. A natural extension is to replace the LSTM with a Transformer-based next-stage predictor that uses self-attention to capture longer-range dependencies in the attacker's behaviour and may produce more diverse stage trajectories. Recent surveys (Gueriani; Kheddar; Mazari, 2023) suggest this is a productive direction for next-generation IoT-defense research.

Direction 5 — Generative augmentation for OOD generalisation. Cybersecurity datasets, including CICIoT2023, are highly imbalanced, with rare but critical attack signatures. A Generative Adversarial Network (or modern equivalent) could synthesise high-fidelity feature vectors for under-represented attack classes, with explicit class-conditional and stage-conditional sampling, to close the supervised-stage-detector recall gap at the source. This direction is more ambitious than F14 (Direction 3) because it produces *new* training data rather than perturbations of held-out data.

Direction 6 — Multi-agent and federated extensions. A real IoT deployment includes many overlapping defenders (e.g. a per-segment defender plus a central SOC), which can be modelled as a cooperative multi-agent system in which specialised agents defend different network segments and learn to coordinate. Federated extensions would allow defender policies to be trained on diverse network deployments without centralising sensitive traffic data. Both extensions inherit the kill-chain decision frame and the audit-first reproducibility protocol from this dissertation.

References

Alam, M. M.; Jahan, I.; Wang, W. IoTWarden: A Deep Reinforcement Learning Based Real-time Defense System to Mitigate Trigger-action IoT Attacks. **arXiv preprint arXiv:2401.08141**, 2024. arXiv: 2401.08141 [cs.CR].

Chen, W. *et al.* Deep Reinforcement Learning for Internet of Things: A Comprehensive Survey. **IEEE Communications Surveys & Tutorials**, v. 23, n. 3, p. 1659–1692, 2021. DOI: 10.1109/COMST.2021.3073036.

Al-Garadi, M. A. *et al.* A Survey of Machine and Deep Learning Methods for Internet of Things (IoT) Security. **IEEE Communications Surveys & Tutorials**, v. 22, n. 3, p. 1646–1685, 2020. DOI: 10.1109/COMST.2020.2988293.

Guan, C. *et al.* HoneyIoT: Adaptive High-Interaction Honeypot for IoT Devices Through Reinforcement Learning. *In:* PROCEEDINGS of the 16th ACM Conference on Security and Privacy in Wireless and Mobile Networks (WiSec '23). Guildford, United Kingdom: ACM, 2023. DOI: 10.1145/3558482.3590195.

Gueriani, A.; Kheddar, H.; Mazari, A. C. Deep Reinforcement Learning for Intrusion Detection in IoT: A Survey. *In:* 2023 2nd International Conference on Electronics, Energy and Measurement (IC2EM 2023). [*S. l.: s. n.*], 2023. Also available as arXiv preprint arXiv:2405.20038.

Hochreiter, S.; Schmidhuber, J. Long Short-Term Memory. **Neural Computation**, v. 9, n. 8, p. 1735–1780, 1997. DOI: 10.1162/neco.1997.9.8.1735.

Ibitoye, O.; Shafiq, O.; Mativenga, M. A Survey on Anomaly Detection in IoT Networks: A Systematic Approach. **IEEE Access**, v. 9, p. 156157–156185, 2021. DOI: 10.1109/ACCESS.2021.3129598.

Khraisat, A.; Gondal, I.; Vamplew, P.; Kamruzzaman, J. Survey of intrusion detection systems: techniques, datasets and challenges. **Cybersecurity**, v. 2, n. 1, p. 20, 2019. DOI: 10.1186/s42400-019-0038-7.

Mavroeidis, V. A Survey of the Internet of Things (IoT) Security: An Overview of Attacks and Defenses. **Computers**, v. 12, n. 3, p. 49, 2023. DOI: 10.3390/computers12030049.

Mitchell, M. *et al.* Model Cards for Model Reporting. *In:* PROCEEDINGS of the Conference on Fairness, Accountability, and Transparency (FAT*'19). [*S. l.*]: ACM, 2019. p. 220–229. DOI: 10.1145/3287560.3287596.

Mnih, V.; Badia, A. P., *et al.* Asynchronous Methods for Deep Reinforcement Learning. *In:* Balcan, M.-F.; Weinberger, K. Q. (eds.). **Proceedings of the 33rd International Conference on Machine Learning (ICML'16)**. [*S. l.: s. n.*], 2016. v. 48. (PMLR), p. 1928–1937.

Mnih, V.; Kavukcuoglu, K., *et al.* Human-level control through deep reinforcement learning. **Nature**, v. 518, n. 7540, p. 529–533, 2015. DOI: 10.1038/nature14236.

Morgan, S. **Official Cybercrime Report 2025**. [*S. l.: s. n.*], May 2025. https://cybersecurityventures.com/official-cybercrime-report-2025/. Accessed on: 2025-07-12.

Neto, E. C. P. *et al.* CICIoT2023: A Real-Time Dataset and Benchmark for Large-Scale Attacks in IoT Environment. **Sensors**, v. 23, n. 13, p. 5941, 2023. DOI: 10.3390/s23135941.

Nguyen, T. G. *et al.* Federated Deep Reinforcement Learning for Traffic Monitoring in SDN-Based IoT Networks. **IEEE Transactions on Cognitive Communications and Networking**, v. 7, n. 4, p. 1048–1065, 2021. DOI: 10.1109/TCCN.2021.3117326.

Schulman, J. *et al.* Proximal Policy Optimization Algorithms. **arXiv preprint arXiv:1707.06347**, 2017. arXiv: 1707.06347 [cs.LG].

Sutton, R. S.; Barto, A. G. **Reinforcement Learning: An Introduction**. 2nd. [*S. l.*]: The MIT Press, 2018.

Tawalbeh, L.; Muheidat, F.; Tawalbeh, M.; Quwaider, M. IoT security and privacy: A review. **Journal of King Saud University - Computer and Information Sciences**, v. 32, n. 2, p. 156–164, 2020. DOI: 10.1016/j.jksuci.2018.07.005.

Tharewal, S. *et al.* Intrusion Detection System for Industrial Internet of Things Based on Deep Reinforcement Learning. **Wireless Communications and Mobile Computing**, v. 2022, p. 8, 2022. Article ID 9023719. DOI: 10.1155/2022/9023719.

Uprety, A.; Rawat, D. B. Reinforcement Learning for IoT Security: A Comprehensive Survey. **IEEE Internet of Things Journal**, v. 8, n. 11, p. 8693–8706, 2021. DOI: 10.1109/JIOT.2020.3040957.

Vailshery, L. S. **Number of Internet of Things (IoT) connections worldwide**. [*S. l.: s. n.*], June 2025. https://www.statista.com/statistics/1183457/iot-connected-devices-worldwide/. Accessed on: 2025-07-11.

Yang, W.; Acuto, A.; Zhou, Y.; Wojtczak, D. A Survey for Deep Reinforcement Learning Based Network Intrusion Detection. **arXiv preprint arXiv:2410.07612**, 2024. arXiv: 2410.07612 [cs.CR].

A Reproducibility Protocol, Manifest Hash Chain, and Verification Recipe

This appendix documents the audit-first reproducibility protocol that underpins every numerical claim in this dissertation, following a model-card-style template inspired by the documentation discipline introduced for machine-learning artefacts (Mitchell *et al.*, 2019). The protocol is implemented as a SHA-256 hash chain pinning every figure, table, and benchmark number to its inputs and the producing Git commit, and as a smoke-reproducibility harness that verifies the chain end-to-end on a fresh checkout in approximately five seconds.

A.1 Per-Phase Manifest Pattern

Every phase of the framework emits one or more `manifest.json` files under `docs/results/<phase>/` that record (a) the SHA-256 hash of every input artefact (data splits, trained models, scaler, episode roll-outs, upstream phase manifests), (b) the producing Git commit (`git_sha`), and (c) the producing-script invocation (`produced_by`). Each manifest also records the SHA-256 hash of every output artefact (figure, summary JSON, scoreboard) so that downstream phases can pin the upstream output by hash, not by filename alone. The per-phase artefact inventory is summarised in Table 5.

Table 5: Per-phase reproducibility-artefact inventory under `docs/results/`. `F<k>` entries are figures or tables, `G<n>_scoreboard.json` are gate-evaluation scoreboards. Run-time roll-out JSONLs (under `runs/<phase>/`) are gitignored but their SHA-256 hashes are pinned by the phase manifest.

Phase	Path	Key artefacts
1	`01_dataset/`	`F0_class_distribution.png`, `F0_stage_distribution.png`, `F0_`
2	`02_red_team/`	`F1_learning_curves.png`, `F2_transition_matrix_comparison.`
3	`03_env/`	`PLAN.md`, `RESULTS.md` (no figures; environment is infrastructure)
4	`04_detector/`	`F11_per_stage_recall.png`, `F11_summary.json`, `G4_scoreboard`
5	`05_blue_team/`	`F3/F4_*.png`, `F3/F4_summary.json`, `F3/F4_manifest.json`, `T1_h`
6	`06_benchmark/`	`F5/F6/F7/F8_*.png`, per-figure `summary.json` + `manifest.json` +
7	`07_ablation/`	`F9/F10/F12/F15_*.png`, per-figure `summary.json` + `manifest.jso`

A.2 Gate Scoreboards

Four G<n>_scoreboard.json files (one per gated phase) record the threshold-vs.-observed value for every exit gate, the verdict bucketed into PASS / PASS-WITH-FINDING / PASS-WITHOUT-STRETCH / FAIL-WITH-FINDING / FAIL / SKIP, and a finding-id cross-reference where the verdict is contingent on a pre-registered finding-activation rule. Schema version 2.0 (Step-8 unification) is used uniformly across G4/G5/G6/G7. The verdict tally at the head of the dissertation's evaluation chain (commit 26b8df2, the post-Step-8 main) is summarised in Table 6.

Table 6: Gate-scoreboard summary across the four gated phases. Verdicts are reported per-gate; the dissertation's headline-finding gates (G6.2 audit-AF2 reframe, G5.4 reward-hacking, G7.2 reward-component sweep, G7.9 OOD finding-activation) are highlighted.

Phase	PASS	PASS-W/-FINDING	PASS-W/O-STRETCH	FAIL-W/-FINDING	Headline-gate ID
4 (G4)	4	1	—	—	G4.4 (OOD recall asymmetr
5 (G5)	6	1	—	—	G5.4 (reward-hacking, mit-r
6 (G6)	4	1	—	1	G6.2 (audit-AF2 reframe, 82
7 (G7)	7	—	1	2	G7.2 (F9 impact_is_term

A.3 Smoke-Reproducibility Harness

The harness scripts/reproducibility_smoke.py is the canonical end-to-end verification recipe. It walks every committed manifest.json file under docs/results/ and verifies, for every recorded input SHA-256, that the on-disk SHA-256 of that input *right now* matches the recorded one. Output is bucketed into four categories:

OK The recorded input SHA matches the on-disk SHA. The expected case for every input.

FAIL The recorded input SHA does *not* match the on-disk SHA, and the divergence is not on the harness's pre-registered known-divergences list. This indicates that an input artefact has changed since the producing manifest was emitted, and the figure or scoreboard pinned by that manifest is no longer reproducible-by-hash from the inputs on disk. FAIL verdicts must be triaged.

KNOWN-DIVERGENCE The recorded SHA does not match on-disk *and* the divergence is documented and audited. The harness ships with a _KNOWN_DIVERGENCES table that explicitly enumerates every such case with its finding-id and explanatory note. The two pre-registered divergences in this dissertation are the Phase-1 and Phase-2 manifests'

recorded splits-manifest SHA (`82aa1214...`, the pre-leakage-fix manifest) versus the on-disk post-fix SHA (`1e99d596...`); both are documented in `docs/results/02_red_team/RE` §5.3.

SKIP An input declared by the manifest is gitignored and not on disk on this checkout. Typical case: large feature arrays (`features.npy`, `stages.npy`) and split index files that are derived from the raw CICIoT2023 dataset and therefore not committed.

The harness's verdict at the head of the dissertation's evaluation chain (commit `26b8df2`) is **PASS**: **458 OK / 0 FAIL / 2 KNOWN-DIVERGENCE / 6 SKIP**. A non-zero FAIL count would block merge of any commit that introduces it.

A.4 Verification Recipe (Fresh-Checkout Reproduction)

On a fresh checkout, the empirical record can be re-verified end-to-end without retraining any model:

```
git clone <repo-url> rl-iot-defense-system
cd rl-iot-defense-system
python -m venv .venv && source .venv/bin/activate
pip install -r requirements.txt
pytest -q                                          # expect: 411 passed
python -m scripts.reproducibility_smoke          # expect: VERDICT PASS
                                                   #         458 OK / 0 FAIL
                                                   #         2 KNOWN-DIVERGENC
```

A full re-train of every phase from scratch (preserving the audited reproducibility contract) is also supported via `make`:

```
make phase-1                          # CICIoT2023 splits + F0a/F0b
                                      # (requires raw dataset on disk)
make phase-2                          # LSTM red team + F1/F2
make phase-4                          # supervised stage detector + F11
make phase-5-sweep PHASE5_TIMESTEPS=250000   # ~108 min CPU
make phase-6                          # held-out benchmark + F5/F6/F7/F8
make phase-7                          # ablations + F9/F10/F12/F15 (~7.5 h CPU)
```

```
make phase-7-close                    # G7 scoreboard + RESULTS.md
```

After a full re-train the per-phase `manifest.json` files are re-emitted with new input SHAs corresponding to the freshly trained checkpoints; the harness will then verify the new chain end-to-end.

A.5 Hardware and Environment

All experiments reported in this dissertation were run on a single CPU core. No GPU is required for any phase (the LSTM red team, the MLP stage detector, and the three DRL agents all train within minutes-to-hours on commodity CPU). Software environment: Python 3.9, PyTorch (CPU build), scikit-learn, Stable Baselines3, Gymnasium. Total wallclock for the full audited evaluation chain (Phases 1–7) is approximately 8.5 h on a single CPU core, dominated by the Phase-7 ablation sweeps; the held-out benchmark (Phase 6) and the smoke-reproducibility harness together take less than 11 minutes.

B CICIoT2023 Label to Kill Chain Stage Mapping

The 34 CICIoT2023 labels (33 attack classes plus `BenignTraffic`) are projected onto the five Kill Chain stages defined in Section 2.1 via the closed mapping in Table 7. The mapping is implemented in `src/utils/label_mapper.py::AbstractStateLabelMapper`; an unknown label encountered at processing time raises an explicit `KeyError` so that any new attack class added to the dataset cannot silently leak into an existing stage. The full design rationale, including why the Lockheed Martin seven-step chain is coarsened to five stages here and why the BENIGN → RECON → ACCESS → MANEUVER → IMPACT ordering is monotone in attacker progress, is provided in `docs/kill-chain-mapping.md`.

C Hyperparameters per Algorithm

The Phase-5 hyperparameter table (T1) for DQN, PPO, and A2C is reproduced in Table 8 from `docs/results/05_blue_team/T1_hparams.json`, which is the canonical hash-pinned source. All values are Stable Baselines3 defaults, frozen for the thesis at PLAN §8 D5.4; the same values are used across the five seeds (`seed` $\in \{0, 1, 2, 3, 4\}$) and the 250,000 environment

Table 7: Per-class CICIoT2023 to Kill Chain stage mapping. Classes marked ⋆ are reserved as held-out out-of-distribution attack classes (Section 3.2.3) and are never seen during training of either the supervised stage detector or the DRL agents.

Stage	CICIoT2023 labels
BENIGN	`BenignTraffic`
RECON	`Recon-PortScan`, `Recon-OSScan`, `Recon-HostDiscovery`, `Recon-PingSweep`, `VulnerabilityScan`⋆
ACCESS	`BrowserHijacking`, `CommandInjection`, `SqlInjection`, `Backdoor_Malware`, `Uploading_Attack`, `XSS`⋆, `DictionaryBruteForce`
MANEUVER	`MITM-ArpSpoofing`, `DNS_Spoofing`, `Mirai-greeth_flood`, `Mirai-greip_flood`
IMPACT	DoS variants (`DoS-TCP_Flood`, `DoS-UDP_Flood`, `DoS-SYN_Flood`, `DoS-HTTP_Flood`); DDoS variants (`DDoS-TCP_Flood`, `DDoS-UDP_Flood`, `DDoS-SYN_Flood`, `DDoS-RSTFINFlood`, `DDoS-PSHACK_Flood`, `DDoS-SynonymousIP_Flood`, `DDoS-ICMP_Flood`, `DDoS-ICMP_Fragmentation`, `DDoS-UDP_Fragmentation`, `DDoS-ACK_Fragmentation`, `DDoS-SlowLoris`, `DDoS-HTTP_Flood`⋆); Mirai impact (`Mirai-udpplain`⋆)

steps per run. Empty cells indicate hyperparameters that do not apply to the algorithm (e.g. off-policy DQN-only fields are blank for the on-policy PPO and A2C rows).

Table 8: Phase-5 per-algorithm hyperparameters. Values are frozen at PLAN §8 D5.4 and used across all five seeds and 250,000 environment steps per run. Empty cells indicate hyperparameters that are not defined for the algorithm.

algo	lr	n_{steps}	γ	λ_{GAE}	ent_coef	vf_coef	batch_size	n_{epochs}
A2C	$7{\cdot}10^{-4}$	5	0.99	1.00	0.000	0.50	—	—
DQN	$1{\cdot}10^{-3}$	—	0.99	—	—	—	32	—
PPO	$3{\cdot}10^{-4}$	2048	0.99	0.95	0.010	0.50	64	10

For DQN-specific off-policy hyperparameters (replay buffer, exploration schedule, target-network update interval), the values used were: `buffer_size` = 50,000, `learning_starts` = 1000, `tau` = 1.0, `target_update_interval` = 1000, `exploration_fraction` = 0.10, `exploration_initial_eps` = 1.0, `exploration_final_eps` = 0.05, `max_grad_norm` = 0.5. The complete machine-readable record is at `docs/results/05_blue_team/T1_hparams.jso`

The Phase-7 ablations re-use the PPO hyperparameters above unchanged — only the environment (`impact_is_terminal`, `p_defender_deescalation`) and the reward coeffi-

cients (`defense_success_bonus`, `penalty_missed_impact`, `reward_proportional`, `reward_benign_passive`, `penalty_disproportionate`) are perturbed.